RKLLM3 Server User Guide

Document ID: RK-YH-YF-436

Version: V1.2.0

Date: 2026-10-23

Confidentiality: □Top Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, Software Boulevard, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides a detailed guide to using RKLLM3 Server, covering service startup, API invocation, Session/Slot management, KV Cache, model extensions, and multimodal input. RKLLM3 Server is an LLM HTTP Server built on RKNPU3 (Rockchip NPU), providing clients with an OpenAI-compatible HTTP API.

Intended Audience

This document is intended for the following engineers:

- Technical Support Engineers

- Software Development Engineers

Revision History

Version	Author	Date	Description	Reviewer
V1.1.0	HPC	2026-08-06	Initial release	熊伟
V1.2.0	HPC	2026-10-23	1. Improved multi-card cascade inference 2. Support LLM structured output (Grammar) 3. Support device-side model loading 4. Support loading multiple LoRA weight files 5. Added macOS arm64 and Linux x86_64 platform support	熊伟

Table of Contents

RKLLM3 Server User Guide

1. RKLLM3 Server Overview

1.1 Use Cases

1.2 System Architecture

1.3 Supported Platforms

2. Installation and Startup

2.1 Prerequisites

2.1.1 Environment Requirements

2.1.2 Model Preparation

2.2 Model and Configuration Preparation

2.2.1 Model Directory Structure

2.2.2 Parameter Reference

2.2.3 Configuration File

2.3 Starting the Service

2.3.1 Command-Line Startup

2.3.2 Configuration File Startup

2.3.3 Common Startup Parameters

2.3.4 Encrypted Model Loading

2.3.5 Device-Side Model Path Loading

2.4 Health Check

2.4.1 Health Check Endpoint

2.4.2 Service Status Verification

2.5 Logging and Startup Issues

2.5.1 Logging

2.5.2 Common Startup Issues

3. Quick Start

3.1 curl Invocation

3.1.1 Request Endpoint

3.1.2 Example

3.2 Python Client

3.2.1 SDK Usage

3.2.2 Examples

3.3 Streaming and Non-Streaming Requests

3.3.1 Feature Overview

3.3.2 Request Endpoint
3.3.3 Examples
3.4 Result Validation
3.4.1 Response Format
3.4.2 Output Validation
4. Session (Slot) Management
4.1 Multi-Session
4.1.1 Feature Overview
4.1.2 API
4.1.3 Example
4.1.4 Usage Constraints
4.2 Slot Lifecycle
4.3 Pause and Resume
4.3.1 Feature Overview
4.3.2 API
4.3.3 Example
4.4 Interrupt
4.4.1 Feature Overview
4.4.2 API
4.4.3 Example
4.5 Concurrency Scheduling Mechanism
4.5.1 Feature Overview
4.5.2 Concurrency Scheduling Model
4.5.3 Slot Allocation Rules
4.5.4 Response to Pause/Resume/Interrupt
5. KV Cache
5.1 Save and Load
5.1.1 Feature Overview
5.1.2 API
5.1.3 Usage Constraints
5.1.4 Examples
5.2 Checkpoint
5.2.1 Feature Overview
5.2.2 API
5.2.3 Examples

6. Model Extensions
    6.1 LoRA
        6.1.1 Feature Overview
        6.1.2 API
    6.2 Tie Word Embedding
        6.2.1 Feature Overview
        6.2.2 API
        6.2.3 Examples
    6.3 Function Calling
        6.3.1 Feature Overview
        6.3.2 API
        6.3.3 Examples
    6.4 Embedding Model
        6.4.1 Feature Overview
        6.4.2 API
        6.4.3 Example
7. Multimodal Input
    7.1 Image Input
        7.1.1 Feature Overview
        7.1.2 API
        7.1.3 Example
    7.2 Audio Input
        7.2.1 Feature Overview
        7.2.2 API
        7.2.3 Example
    7.3 Video Input
        7.3.1 Feature Overview
        7.3.2 API
        7.3.3 Example
    7.4 Multimodal Request Format
        7.4.1 Request Structure
        7.4.2 Content Types
        7.4.3 Example
8. Multi-Card Cascade Inference
    8.1 Feature Overview

8.2 Configuration Parameters
8.3 Startup Examples
8.4 KV Cache Save & Load (Multi-Card)
9. Grammar Constraints
9.1 Request Fields and Priority
9.2 EBNF Grammar Constraint
9.2.1 Introduction
9.2.2 API
9.2.3 Examples
9.3 JSON Schema Structured Output
9.3.1 Introduction
9.3.2 API
9.3.3 Examples
9.4 Enabling the Grammar Feature and Configuring a Global Grammar
9.4.1 Introduction
9.4.2 API
9.4.3 Examples
9.5 structural_tag Structured Constraint
9.5.1 Introduction
9.5.2 Syntax Structure
9.5.3 Examples
9.6 Combining Structured Output with Tool Calling
9.6.1 Introduction
9.6.2 Combination Rules
9.6.3 Examples
9.7 Combining Structured Output with Thinking Mode
9.7.1 Introduction
9.7.2 The thinking_grammar Field
9.7.3 Examples
9.7.4 Notes
Appendix
A. API Endpoint Index
B. OpenAI Compatibility
B1. POST /v1/chat/completions: Chat Completions
C. Error Codes
C1. Error Type Quick Reference

C2. Common Error Messages and Troubleshooting

D. Configuration Summary

D1. Server Configuration

D2. Model Configuration

1. RKLLM3 Server Overview

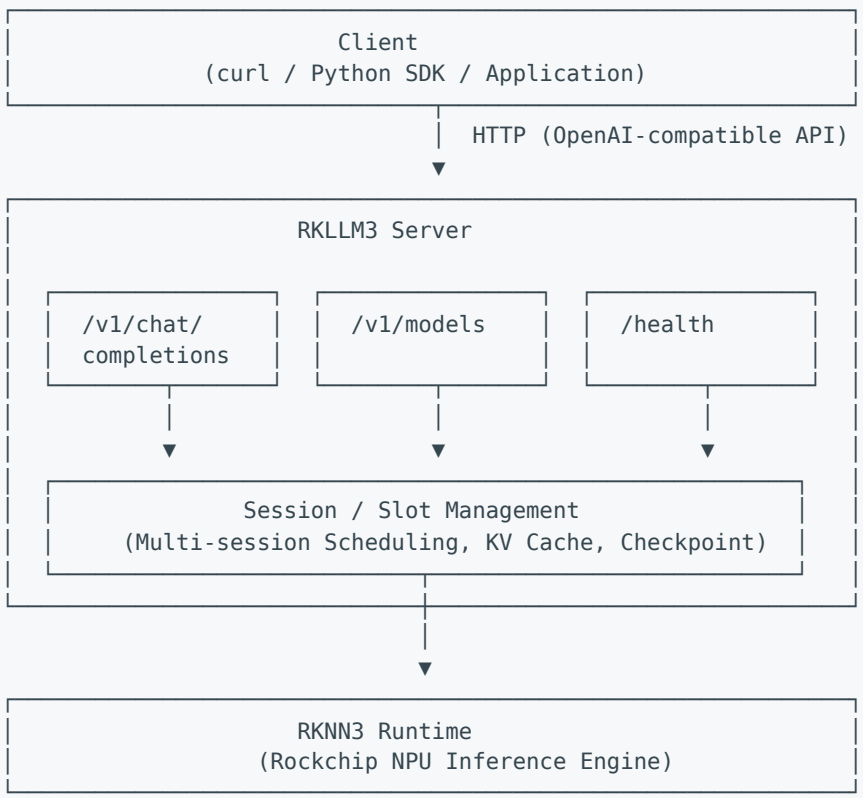
1.1 Use Cases

RKLLM3 Server is an LLM HTTP Server built on RKNPU3 (Rockchip NPU), providing clients with an OpenAI-compatible HTTP API. It is suitable for the following scenarios:

- Deploying large language model inference services on Rockchip platforms
- Providing OpenAI-compatible APIs (chat completions, embeddings, etc.) for upper-layer applications
- Supporting multimodal models (image, audio, and video understanding)
- Multi-user concurrent service with session isolation via the Session (Slot) mechanism

1.2 System Architecture

The core architecture of RKLLM3 Server consists of three layers:



- **Client**: Sends OpenAI-compatible API requests via HTTP
- **RKLLM3 Server**: Receives and parses requests, manages Session scheduling, and coordinates inference tasks
- **RKNN3 Runtime**: Underlying NPU inference engine that executes model inference computations

1.3 Supported Platforms

RKLLM3 Server supports the following platforms:

Platform	**Architecture**
Linux	aarch64
Linux	x86_64
Android	arm64-v8a
Windows	x86_64
macOS	arm64

2. Installation and Startup

2.1 Prerequisites

2.1.1 Environment Requirements

RKLLM3 Server requires the following environment:

- Rockchip NPU device: RK3588/RK3576 boards, or Windows/macOS/Linux x86_64 PCs connected with RK1820/1828 coprocessor; RK3572
- RKNN3 Runtime dynamic libraries
- Converted RKNN model files

2.1.2 Model Preparation

RKLLM3 Server requires the following model files:

File Type	Description	Required
LLM Model (`.rknn`)	Main language model file, specified via `-m`	Required
LLM Weights (`.weight`)	Model weight file, specified via `--weight`	Required
Vision Model (`.rknn`)	Multimodal vision model, specified via `--model2`	Required for vision
Vision Weights (`.weight`)	Vision model weights, specified via `--weight2`	Required for vision
Audio Model (`.rknn`)	Multimodal audio model, specified via `--model3`	Required for audio
Audio Weights (`.weight`)	Audio model weights, specified via `--weight3`	Required for audio
Tokenizer (`.gguf`)	Vocabulary file, specified via `--vocab`	Required
Embed (`.bin`)	Embedding weight file, specified via `--embed`	Required for non-tie-word mode
Embed PLE (`.bin`)	Per-layer inputs embed file for Gemma4 models	Required for Gemma4
Rope Tensor (`.safetensors`)	Rope cache file for Gemma4 models	Required for Gemma4
Mel Filter (`.txt`)	Mel filters file for audio models, specified via `--mel-filter`	Required for some audio models

2.2 Model and Configuration Preparation

2.2.1 Model Directory Structure

It is recommended to organize model files as follows:

```
models/
├── qwen2.5-3b/
│   ├── qwen2.5-3b.rknn
│   ├── qwen2.5-3b.weight
│   ├── qwen2.5-3b.tokenizer.gguf
│   └── qwen2.5-3b.embed.bin
├── qwen3-vl-2b/
│   ├── Qwen3-VL-2B-llm.rknn
│   ├── Qwen3-VL-2B-llm.weight
│   ├── Qwen3-VL-2B-vision.rknn
│   ├── Qwen3-VL-2B-vision.weight
│   ├── Qwen3-VL-2B-llm.tokenizer.gguf
│   └── Qwen3-VL-2B-llm.embed.bin
└── params.json
```

2.2.2 Parameter Reference

RKLLM3 Server supports starting the HTTP service via command-line arguments.

General Parameters

Parameter	Description
`-a, --alias STRING`	Set model alias (for REST API use)
`-c, --ctx-size N`	Prompt context size (default: 4096, 0 = load from model). Supports `#` delimiter for per-slot values
`-n, --predict, --n-predict N`	Number of tokens to predict (default: -1, -1 = context size). Supports `#` delimiter for per-slot values
`-m, --model FNAME`	LLM model path, supports the `dev:` prefix for device-side paths (see section 2.3.5)
`--weight FNAME`	LLM weight path (optional, auto-derived from the model path if not set), supports the `dev:` prefix for device-side paths
`--model2 FNAME`	Vision model path for multimodal models, supports the `dev:` prefix for device-side paths
`--weight2 FNAME`	Vision weight path for multimodal models (optional), supports the `dev:` prefix for device-side paths
`--model3 FNAME`	Audio model path for multimodal models, supports the `dev:` prefix for device-side paths
`--weight3 FNAME`	Audio weight path for multimodal models (optional), supports the `dev:` prefix for device-side paths
`--core-mask MASK`	NPU core mask for LLM model execution, in hex or decimal. Each bit controls one NPU core (e.g., `0x01` = core 0 only, `0x03` = cores 0+1, `0xff` = all cores). Default: auto (use all available cores)
`--core-mask2 MASK`	NPU core mask for vision model (model2) execution. Default: auto (use all available cores)
`--core-mask3 MASK`	NPU core mask for audio model (model3) execution. Default: auto (use all available cores)
`--vocab FNAME`	Vocabulary file path

Parameter	Description
`--embed FNAME`	Embedding bin file path
`--embed-ple FNAME`	Per-layer inputs embed bin file path for Gemma4 models
`--rope-tensor FNAME`	Rope cache safetensors file path for Gemma4 models
`--mel-filter FNAME`	Mel filters path (for audio models)
`--lora-weight FNAME`	LLM LoRA weight path. Can be passed multiple times to load multiple independent `.lora_weight` files; each file can contain multiple adapters shared across all sessions; scales can be set per-session via the `/lora-adapters` API
`--decrypt-key, --key FNAME`	RSA-encrypted LLM model user key package path (required when loading encrypted models)
`--decrypt-key2, --key2 FNAME`	RSA-encrypted Vision model user key package path (for encrypted multimodal vision models)
`--decrypt-key3, --key3 FNAME`	RSA-encrypted Audio model user key package path (for encrypted multimodal audio models)
`--img-start STRING`	Image input prefix for multimodal models
`--img-end STRING`	Image input suffix for multimodal models
`--img-content STRING`	Image input placeholder for multimodal models
`--audio-start STRING`	Audio input prefix for multimodal models
`--audio-end STRING`	Audio input suffix for multimodal models
`--audio-content STRING`	Audio input placeholder for multimodal models
`--video-start STRING`	Video input prefix for qwen3_vl deepstack video mode
`--video-end STRING`	Video input suffix for qwen3_vl deepstack video mode
`--video-content STRING`	Video input placeholder for qwen3_vl deepstack video mode
`--img-width N`	Input image width for multimodal models (required for some trimmed models, e.g., qwen3_vl)
`--img-height N`	Input image height for multimodal models (required for some trimmed models, e.g., qwen3_vl)
`--chat-template-file JINJA_TEMPLATE_FILE`	Set a custom Jinja chat template (default: use template from model metadata)
`--embedding`	Whether this is an embedding model

Parameter	Description
`--reasoning-format FORMAT`	Controls whether thinking tags are allowed in responses and how they are returned. Options: - `none` : Thinking content stays in `message.content` - `deepseek` : Thinking content placed in `message.reasoning_content` (default: `none`)
`--reasoning [on|off|auto]`	Use reasoning/thinking in conversations (`on` , `off` , or `auto` ; default: `auto` , obtained from template)
`--record-path STRING`	Directory for saving user requests (default: empty, not saved)
`--n-session N`	Number of sessions for multi-session scenarios (default: 1, max: 16; consider device memory)

Sampling Parameters

Parameter	Description
`--temp N`	Temperature (default: 0.8)
`--top-k N`	Top-k sampling (default: 1, 0 = disabled)
`--top-p N`	Top-p sampling (default: 0.9, 1.0 = disabled)
`--repeat-penalty N`	Penalize repeated token sequences (default: 1.0, 1.0 = disabled)
`--presence-penalty N`	Repeat alpha presence penalty (default: 0.0, 0.0 = disabled)
`--frequency-penalty N`	Repeat alpha frequency penalty (default: 0.0, 0.0 = disabled)

Server-Specific Parameters

Parameter	Description
`-h, --help, --usage`	Print usage and exit
`--host HOST`	Listening IP address (default: 127.0.0.1)
`--port PORT`	Listening port (default: 8080)
`-to, --timeout N`	Server read/write timeout in seconds (default: 600)
`--device-id STRING`	Device ID (required for multi-device scenarios)
`--log-level N`	Log verbosity level (default: `0`)

2.2.3 Configuration File

RKLLM3 Server supports starting via a JSON configuration file (`params.json`). When `--params-file` is specified, all other command-line arguments are ignored; only parameters in `params.json` take effect.

Basic configuration file format:

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen2.5-0.5B": {
            "alias": "Qwen2.5-0.5B",
            "model": "Qwen2.5-0.5B-Instruct.rknn",
            "weight": "Qwen2.5-0.5B-Instruct.weight",
            "model2": "",
            "weight2": "",
            "model3": "",
            "weight3": "",
            "core-mask": 0,
            "core-mask2": 0,
            "core-mask3": 0,
            "vocab": "Qwen2.5-0.5B-Instruct.tokenizer.gguf",
            "embed": "Qwen2.5-0.5B-Instruct.embed.bin",
            "mel-filter": "",
            "lora-weight": "",
            "decrypt-key": "",
            "decrypt-key2": "",
            "decrypt-key3": "",
            "ctx-size": 1024,
            "predict": 512,
            "temp": 0.8,
            "top-k": 1,
            "top-p": 0.8,
            "repeat-penalty": 1.1,
            "presence-penalty": 1.0,
            "frequency-penalty": 1.0,
            "img-start": "",
            "img-end": "",
            "img-content": "",
            "audio-start": "",
            "audio-end": "",
            "audio-content": "",
            "video-start": "",
            "video-end": "",
            "video-content": "",
            "img-width": 0,
            "img-height": 0,
            "chat-template-file": "",
            "embedding": false,
            "record-path": "",
            "reasoning-format": "none",
            "reasoning": "auto",
            "n-session": 1
        }
    }
}
```


2.3 Starting the Service

2.3.1 Command-Line Startup

The following examples show command-line startup for various scenarios:

Text-Only LLM Model (Standard Mode, requires embed file)

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --repeat-
penalty 1.1 --presence-penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --
temp 0.8
```

LLM Model (Tie-Word Mode, no embed file required)

```
./rkllm3-server -m Qwen3-VL-2B-llm-tie-word.rknn --weight Qwen3-VL-2B-llm-tie-
word.weight --vocab Qwen3-VL-2B-llm_quant.tokenizer.gguf --tie-word-embedding 1 --
host 0.0.0.0 --port 8080 -c 4096 --n-predict 512 --repeat-penalty 1.1 --presence-
penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8
```

Gemma4 Model

```
./rkllm3-server -m gemma-4.rknn --vocab gemma-4.tokenizer.gguf --embed gemma-
4.embed.bin --embed-ple gemma-4_per_layer_inputs.embed.bin --rope-tensor gemma-
4.safetensors --host 0.0.0.0 --port 8080 -c -1 --n-predict 2048 --repeat-penalty 1.0
--presence-penalty 0 --frequency-penalty 0 --top-k 1 --top-p 0.8 --temp 0.8
```

Multimodal Model (LLM + Vision)

```
# MiniCPM-3o
./rkllm3-server -m MiniCPM-3o-llm.rknn --model2 MiniCPM-3o-vision.rknn --vocab
MiniCPM-3o-llm.tokenizer.gguf --embed MiniCPM-3o-llm.embed.bin --host 0.0.0.0 --port
8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-
penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "<image>" --img-end "
</image>" --img-content "<unk>"

# Qwen2.5-VL-3B
./rkllm3-server -m Qwen2.5-VL-3B-llm.rknn --model2 Qwen2.5-VL-3B-vision.rknn --vocab
Qwen2.5-VL-3B-llm.tokenizer.gguf --embed Qwen2.5-VL-3B-llm.embed.bin --host 0.0.0.0
--port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --
frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "
<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --img-
width 392 --img-height 392

# Qwen3-VL-2B
./rkllm3-server -m Qwen3-VL-2B-llm.rknn --model2 Qwen3-VL-2B-vision.rknn --vocab
Qwen3-VL-2B-llm.tokenizer.gguf --embed Qwen3-VL-2B-llm.embed.bin --host 0.0.0.0 --
port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --
frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "
<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --img-
width 384 --img-height 384

# Qwen3.5-4B
./rkllm3-server -m Qwen3.5-4B.rknn --model2 Qwen3.5-4B-vision.rknn --vocab Qwen3.5-
4B.tokenizer.gguf --embed Qwen3.5-4B.embed.bin --host 0.0.0.0 --port 8080 -c 0 --n-
predict 2048 --repeat-penalty 1.0 --presence-penalty 1.5 --frequency-penalty 0 --
top-k 1 --top-p 0.9 --temp 1.0 --img-start "<|vision_start|>" --img-end "
<|vision_end|>" --img-content "<|image_pad|>" --img-width 384 --img-height 384
```

Multimodal Model (LLM + Audio)

```
./rkllm3-server -m gemma-4.rknn --model3 gemma-4-audio.rknn --vocab gemma-
4.tokenizer.gguf --embed gemma-4.embed.bin --embed-ple gemma-
4_per_layer_inputs.embed.bin --rope-tensor gemma-4.safetensors --host 0.0.0.0 --port
8080 -c -1 --n-predict 2048 --repeat-penalty 1.0 --presence-penalty 0 --frequency-
penalty 0 --top-k 1 --top-p 0.8 --temp 0.8 --audio-start "<|audio>" --audio-end "
<audio|>" --audio-content "<|audio|>"
```

Multimodal Model (LLM + Vision + Audio)

```
./rkllm3-server -m Qwen2.5-Omni-3B-llm.rknn --model2 Qwen2.5-Omni-3B-vision.rknn --
model3 Qwen2.5-Omni-3B-audio.rknn --vocab Qwen2.5-Omni-3B-llm.tokenizer.gguf --embed
Qwen2.5-Omni-3B-llm.embed.bin --mel-filter mel_128_filters.txt --host 0.0.0.0 --port
8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-
penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "<|vision_bos|>" --img-end
"<|vision_eos|>" --img-content "<|IMAGE|>" --audio-start "<|audio_bos|>" --audio-end
"<|audio_eos|>" --audio-content "<|AUDIO|>"
```

Embedding Model

```
./rkllm3-server -m Qwen3-Embedding-4B.rknn --vocab Qwen3-Embedding-4B.tokenizer.gguf
--embed Qwen3-Embedding-4B.embed.bin --embedding
```

2.3.2 Configuration File Startup

To load multiple models simultaneously, use `--params-file` to specify a JSON configuration file:

```
./rkllm3-server --params-file params.json
```

Multi-model configuration file example:

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen2.5-0.5B": {
            "alias": "Qwen2.5-0.5B",
            "model": "Qwen2.5-0.5B-Instruct.rknn",
            "weight": "Qwen2.5-0.5B-Instruct.weight",
            "model2": "",
            "weight2": "",
            "model3": "",
            "weight3": "",
            "core-mask": 0,
            "core-mask2": 0,
            "core-mask3": 0,
            "vocab": "Qwen2.5-0.5B-Instruct.tokenizer.gguf",
            "embed": "Qwen2.5-0.5B-Instruct.embed.bin",
            "mel-filter": "",
            "lora-weight": "",
            "decrypt-key": "",
            "decrypt-key2": "",
            "decrypt-key3": "",
            "ctx-size": 1024,
            "predict": 512,
            "temp": 0.8,
            "top-k": 1,
            "top-p": 0.8,
            "repeat-penalty": 1.1,
            "presence-penalty": 1.0,
            "frequency-penalty": 1.0,
            "img-start": "",
            "img-end": "",
            "img-content": "",
            "audio-start": "",
            "audio-end": "",
            "audio-content": "",
            "video-start": "",
            "video-end": "",
            "video-content": "",
            "img-width": 0,
            "img-height": 0,
            "chat-template-file": "",
            "embedding": false,
            "record-path": "",
            "reasoning-format": "none",
            "reasoning": "auto",
            "n-session": 1
        },
        "Qwen3-0.6B": {
            "alias": "Qwen3-0.6B",
            "model": "Qwen3-0.6B.rknn",
            "weight": "Qwen3-0.6B.weight",
            "model2": "",
            "weight2": "",
            "model3": "",
            "weight3": "",
            "core-mask": 0,
            "core-mask2": 0,
            "core-mask3": 0,
            "vocab": "Qwen3-0.6B.tokenizer.gguf",
```

```
            "embed": "Qwen3-0.6B.embed.bin",
            "mel-filter": "",
            "lora-weight": "",
            "decrypt-key": "",
            "decrypt-key2": "",
            "decrypt-key3": "",
            "ctx-size": 1024,
            "predict": 512,
            "temp": 0.8,
            "top-k": 1,
            "top-p": 0.8,
            "repeat-penalty": 1.1,
            "presence-penalty": 1.0,
            "frequency-penalty": 1.0,
            "img-start": "",
            "img-end": "",
            "img-content": "",
            "audio-start": "",
            "audio-end": "",
            "audio-content": "",
            "video-start": "",
            "video-end": "",
            "video-content": "",
            "img-width": 0,
            "img-height": 0,
            "chat-template-file": "",
            "embedding": false,
            "record-path": "",
            "reasoning-format": "none",
            "reasoning": "auto",
            "n-session": 1
        }
    }
}
```


2.3.3 Common Startup Parameters

The following are the most commonly used parameter combinations:

Scenario	Minimum Parameter Set
Text-Only LLM	`-m <model> --vocab <vocab> --embed <embed>`
Tie-Word LLM	`-m <model> --vocab <vocab> --tie-word-embedding 1`
LLM + Vision	Above plus `--model2 <vision>` `--img-start` `--img-end` `--img-content`
LLM + Audio	Above plus `--model3 <audio>` `--audio-start` `--audio-end` `--audio-content`
Embedding Model	`-m <model> --vocab <vocab> --embed <embed> --embedding`
Encrypted Model	Above plus `--decrypt-key` (and `--decrypt-key2` / `--decrypt-key3` as needed)
Device-Side Model	`-m <model> --vocab <vocab> --embed <embed>` with a `dev:` prefix on the model path (see section 2.3.5)

2.3.4 Encrypted Model Loading

RKLLM3 Server supports loading RSA-encrypted model files. Encrypted model files typically have the extension `.rknn.enc` and require the corresponding user key package file (`.key`).

Parameter Description

Encrypted model loading uses the following parameters to specify key package paths:

Parameter	Corresponding Model	Description
`--decrypt-key, --key`	LLM model (model)	Must be used together with the encrypted LLM model file
`--decrypt-key2, --key2`	Vision model (model2)	Required only when the Vision model is also encrypted
`--decrypt-key3, --key3`	Audio model (model3)	Required only when the Audio model is also encrypted

Command-Line Examples

```
# Load encrypted model (LLM text-only)
./rkllm3-server -m Qwen3-4B.rknn.enc --decrypt-key Qwen3-4B.key --vocab Qwen3-4B.tokenizer.gguf --embed Qwen3-4B.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8

# Load encrypted model (LLM+VISION multimodal)
./rkllm3-server -m Qwen3-VL-4B-llm.rknn.enc --model2 Qwen3-VL-4B-vision.rknn.enc --decrypt-key Qwen3-VL-4B-llm.key --decrypt-key2 Qwen3-VL-4B-vision.key --vocab Qwen3-VL-4B-llm.tokenizer.gguf --embed Qwen3-VL-4B-llm.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --img-width 384 --img-height 384 --video-start "<|vision_start|>" --video-end "<|vision_end|>" --video-content "<|video_pad|>"

# Load encrypted model (LLM+VISION+AUDIO full multimodal)
./rkllm3-server -m Qwen3-Omni-4B-llm.rknn.enc --model2 Qwen3-Omni-4B-vision.rknn.enc --model3 Qwen3-Omni-4B-audio.rknn.enc --decrypt-key Qwen3-Omni-4B-llm.key --decrypt-key2 Qwen3-Omni-4B-vision.key --decrypt-key3 Qwen3-Omni-4B-audio.key --vocab Qwen3-Omni-4B-llm.tokenizer.gguf --embed Qwen3-Omni-4B-llm.embed.bin --mel-filter mel_128_filters.txt --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --audio-start "<|audio_start|>" --audio-end "<|audio_end|>" --audio-content "<|audio_pad|>"
```

Configuration File Method

When loading encrypted models via `params.json` , add the `decrypt-key` , `decrypt-key2` , and `decrypt-key3` fields to the corresponding model configuration:

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen3-4B-encrypted": {
            "alias": "Qwen3-4B-encrypted",
            "model": "Qwen3-4B.rknn.enc",
            "decrypt-key": "Qwen3-4B.key",
            "vocab": "Qwen3-4B.tokenizer.gguf",
            "embed": "Qwen3-4B.embed.bin",
            "ctx-size": 768,
            "predict": 512
        }
    }
}
```

2.3.5 Device-Side Model Path Loading

RKLLM3 Server supports loading model files from the device-side filesystem (e.g. eMMC). Add a `dev:` prefix to the model or weight path, e.g. `dev:/data/llm.rknn`. With the `dev:` prefix, the server calls the device-side loading API (`rknn3_load_model_from_device`) and skips host-side file existence checks.

Command-Line Example

```
# LLM model and weight on the device-side filesystem
./rkllm3-server -m dev:/data/qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --
embed qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --top-k
1 --top-p 0.8 --temp 0.8
```

Configuration File Example

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen2.5-0.5B": {
            "alias": "Qwen2.5-0.5B",
            "model": "dev:/data/Qwen2.5-0.5B-Instruct.rknn",
            "weight": "dev:/data/Qwen2.5-0.5B-Instruct.weight",
            "vocab": "Qwen2.5-0.5B-Instruct.tokenizer.gguf",
            "embed": "Qwen2.5-0.5B-Instruct.embed.bin",
            "ctx-size": 1024,
            "predict": 512
        }
    }
}
```

Usage Notes

- **Automatic weight derivation**: When `--weight`, `--weight2`, or `--weight3` is not set, the weight path is derived from the model path by replacing `.rknn` with `.weight`; the

`dev:` prefix is preserved. For example, `-m dev:/data/llm.rknn` derives the weight `dev:/data/llm.weight`.

- **Multi-card cascade inference**: Only the first segment (`_seg0`) path needs to be specified; remaining segment paths are derived automatically and the `dev:` prefix is preserved as well. See section 8. Multi-Card Cascade Inference.
- **Paths in one pair must be on the same side**: The model path and weight path of the same model must both be device-side or both host-side. Mixing is not supported; when the model path has a `dev:` prefix but the weight path does not (or vice versa), the server logs an error: `model path has "dev:" prefix but weight "<path>" does not, mixing device/host paths is not supported!` (vision/audio models use `model2` / `model3`).
- **Per-model selection**: In multimodal scenarios each model can independently choose its side. For example, the LLM model uses a `dev:` prefix (device-side) while the Vision/Audio models use host-side paths.
- **Encrypted models**: Device-side encrypted models are also supported. The user key package (`.key`) is always read from the host-side filesystem; `--decrypt-key`, `--key2`, and `--key3` are not affected by the `dev:` prefix.
- **Log display**: The model path in loading logs includes the `dev:` prefix (e.g. `loading model 'dev:/data/llm.rknn'`); this is normal.

> **Note**: The `dev:` prefix only applies to model and weight paths (`-m/--model`, `--weight`, `--model2`, `--weight2`, `--model3`, `--weight3`). Other file path parameters such as `--vocab`, `--embed`, `--rope-tensor`, and `--mel-filter` do not support it; those files must reside on the host-side filesystem.

2.4 Health Check

2.4.1 Health Check Endpoint

RKLLM3 Server provides a `/health` endpoint for checking service status.

Request

```
GET /health
```

Response

While the model is loading (HTTP 503):

```
{
    "error": {
        "code": 503,
        "message": "Loading model",
        "type": "unavailable_error"
    }
}
```

When the model has loaded and the service is ready (HTTP 200):

```
{
    "status": "ok"
}
```

2.4.2 Service Status Verification

After starting the service, verify it is ready with:

```
curl http://localhost:8080/health
```

If `{"status": "ok"}` is returned, the service is running normally.

2.5 Logging and Startup Issues

2.5.1 Logging

RKLLM3 Server uses the `--log-level N` parameter to control log verbosity (default: `0`):

Value	Log Truncation	Request Body Logging	Prompt Token Logging
`0` (default)	Truncated to 1024 bytes	Non-multimedia requests only (no image/audio)	Truncated to 1024 tokens
`1`	Full, no truncation	Non-multimedia requests only (no image/audio)	Truncated to 1024 tokens
`≥2`	Full, no truncation	All requests (including multimedia base64)	Full output

2.5.2 Common Startup Issues

Parameter Configuration Errors

Issue	Corresponding Log
Invalid or misspelled CLI argument	`error: invalid argument: <bad argument>`
Missing value for argument (e.g., no model path after `-m`)	`expected value for argument`
Invalid `--core-mask` value format	`invalid value for --core-mask: "<value>"`
Config file specified by `--params-file` does not exist	`Can not open the params-file "<path>"!`
`models` dict in `params.json` is empty	`The "models" dict in params-file must have at least one subitem!`
Invalid `reasoning-format` value (only `none`, `deepseek`)	`The "reasoning-format" is invalid!`
Invalid `reasoning` value (only `on`, `off`, `auto`)	`The "reasoning" is invalid!`

Issue	Corresponding Log
Template file specified by `--chat-template-file` does not exist	`failed to open file "<path>"`
Unsupported chat template format	`error: the supplied chat template is not supported`
Both `--embedding` and `--reranking` used simultaneously	`error: either --embedding or --reranking can be specified, but not both`

Model File Issues

Issue	Corresponding Log
No model path specified (`-m` missing)	`model path is empty`
Model file (`.rknn`) does not exist	`the model file "<path>" does not exist!`
Weight file (`.weight`) does not exist	`the model file "<path>" does not exist!`
Device-side model path mixed with host-side weight path (`dev:` prefix mismatch)	`model path has "dev:" prefix but weight "<path>" does not, mixing device/host paths is not supported!`
Device-side model loading failed (`dev:` prefixed path)	`rknn3_load_model_from_device failed! ret=<error code>`
Model loading failed (corrupted or incompatible format)	`rknn3_load_model_from_path failed! ret=<error code>`
Model initialization failed	`rknn3_model_init failed! ret=<error code>`
Failed to query model configuration	`rknn3_query RKNN3_QUERY_LLM_CONFIG failed! ret=<error code>`
Comprehensive model loading failure	`failed to load model, '<path>'`
Embed file (`.bin`) does not exist	`Failed to open embed file: <path>`
Embed file mmap failed	`Failed to mmap file`
Embed-ple file does not exist (Gemma4 model)	`Failed to open embed-ple file: <path>`
Vocabulary file (`.gguf`) missing or malformed	`cannot find tokenizer vocab in model file`
Unknown tokenizer type	`unknown tokenizer: '<type>'`

Device & Runtime

Issue	Corresponding Log
No NPU device found	`no device found`
NPU device detection failed	`rknn3_find_devices fail! ret=<error code>`
Invalid `--device-id` specified	`the device-id "<id>" is invalid! please check it carefully!`
RKNN3 Runtime initialization failed	`rknn3_init fail! ret=<error code>`

Issue	Corresponding Log
RKNN3 Session initialization failed	`Failed to initialize session!`
RKNN3 query failed	`rknn_query fail! ret=<error code>`
Port already in use (`--port`)	`couldn't bind HTTP server socket, hostname: <host>, port: <port>`
HTTP Server initialization failed	`Failed to initialize http server`
HTTP Server startup failed	`Failed to start http server`
Exit during model loading phase	`exiting due to model loading error`

Multimodal Issues

Issue	Corresponding Log
Vision model (model2) input size unknown	`the input image size of model2 is unknow, please set by '--img-width' & '--img-height'!`
Unsupported vision model type	`model2 (<model name>) is not supported!`

Other

Issue	Corresponding Log
`tie_word_embedding=1` in request but model not initialized with it	`WRN: tie_word_embedding=1 requested but model was not initialized with --tie-word-embedding`
Tie-word mode without loaded embed data	`tie_word_embedding requires embedding file, but no embedding data was loaded`
KV Cache full (insufficient context)	`failed to decode the batch: KV cache is full - try increasing it via the context size`

3. Quick Start

3.1 curl Invocation

3.1.1 Request Endpoint

RKLLM3 Server provides an OpenAI-compatible `/v1/chat/completions` endpoint for chat completions.

Request

```
POST /v1/chat/completions
```

3.1.2 Example

Send a chat request using curl:

```
curl --request POST --url http://localhost:8080/v1/chat/completions \
    --header "Content-Type: application/json" \
    --data '{"messages": [{"role": "user", "content": "Hello!"}],"n_predict": 128}'
```

3.2 Python Client

3.2.1 SDK Usage

RKLLM3 Server is compatible with the OpenAI Python SDK. You can use the `openai` library directly.

3.2.2 Examples

Text-Only Chat

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "system", "content": "You are ChatGPT, an AI assistant. Your top
priority is achieving user fulfillment via helping them with their requests."},
        {"role": "user", "content": "Write a limerick about python exceptions"}
    ]
)

print(completion.choices[0].message)
```

Multimodal Chat (Image + Audio)

```python
import base64
from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:8080/v1",
    api_key="sk-no-key-required"
)

def encode_base64(media_path):
    with open(media_path, "rb") as media_file:
        return base64.b64encode(media_file.read()).decode("utf-8")

image_path = "demo.jpg"
audio_path = "demo.wav"     # if audio input exists

base64_image = encode_base64(image_path)
base64_audio = encode_base64(audio_path)    # if audio input exists

completion = client.chat.completions.create(
    model="Qwen2.5-VL",
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image_url",
                    "image_url": {
                        "url": f"{base64_image}",
                    },
                },
                {
                    "type": "input_audio",
                    "input_audio": {
                        "data": f"{base64_audio}",
                        "format": "wav",
                    }
                },  # if audio input exists
                {"type": "text", "text": "Please describe the image."},
            ],
        }
    ],
    stream=True,
    extra_body={
        "n_predict": 256,
        "chat_template_kwargs": {"enable_thinking": False},     # For thinking
models (e.g., Qwen3), disable thinking output this way
        "id_slot": 0,           # When --n-session > 1, use this to switch between
sessions
        # Sampling parameters
        "top_k": 1,
        "top_p": 0.9,
        "temperature": 0.8,
        "repeat_penalty": 1.1,
        "frequency_penalty": 1.0,
        "presence_penalty": 1.0,
    }
)

for chunk in completion:
    delta = chunk.choices[0].delta
    if delta.content:
```

```
            print(delta.content, end='', flush=True)
        if chunk.choices[0].finish_reason == "stop":
            break
    print('')
```

3.3 Streaming and Non-Streaming Requests

3.3.1 Feature Overview

RKLLM3 Server supports two response modes:

- **Non-streaming** (`stream=False`): Returns the complete result once generation is finished
- **Streaming** (`stream=True`): Returns results token-by-token via SSE (Server-Sent Events)

3.3.2 Request Endpoint

Request

```
POST /v1/chat/completions
```

Control the mode via the `stream` parameter.

3.3.3 Examples

Non-Streaming Request (curl)

```
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer no-key" \
    -d '{
        "messages": [
            {
                "role": "system",
                "content": "You are ChatGPT, an AI assistant. Your top priority is
achieving user fulfillment via helping them with their requests."
            },
            {
                "role": "user",
                "content": "Write a limerick about python exceptions"
            }
        ]
    }'
```

Streaming Request (Python)

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "user", "content": "Hello!"}
    ],
    stream=True
)

for chunk in completion:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end='', flush=True)
```

3.4 Result Validation

3.4.1 Response Format

RKLLM3 Server's response format is compatible with the OpenAI API. The basic structure of a non-streaming response:

```
{
    "id": "chatcmpl-xxx",
    "object": "chat.completion",
    "created": 1735142223,
    "model": "Qwen2.5-3B",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "..."
            },
            "finish_reason": "stop"
        }
    ]
}
```

3.4.2 Output Validation

The `finish_reason` field indicates the reason for termination:

Value	Meaning
`stop`	Normal completion
`length`	Truncated due to `max_tokens` / `n_predict` limit
`tool_calls`	Tool call triggered

4. Session (Slot) Management

> **Note**: In this document, "session" and "slot" refer to the same concept: an independent inference slot with its own KV Cache and conversation context. The CLI parameter `--n-session` and JSON field `n-session` control the number of slots; API endpoints use the `/slots/{slot_id}` route, and request parameters use `id_slot`. The two terms are used interchangeably.

4.1 Multi-Session

4.1.1 Feature Overview

The `--n-session` parameter creates multiple independent sessions (slots), each with its own KV Cache and conversation context, supporting concurrent user requests.

When `--n-session` is greater than 1, the server creates the corresponding number of slots. Clients specify which slot to use via the `id_slot` parameter (id starts from 0, range `[0, --n-session - 1]`, default: 0).

Use Cases

- **Multi-user concurrent service**: Multiple users chatting simultaneously, each with independent KV Cache, no interference
- **Conversation isolation**: Different tasks/topics use different slots to avoid context confusion

4.1.2 API

Request

```
POST /v1/chat/completions
```

Specify the slot via the `id_slot` parameter.

Parameter	Type	Required	Description
`id_slot`	int	No	Slot ID to use (when `--n-session` > 1, use this to switch sessions; must be less than `--n-session`, default: 0)

4.1.3 Example

Start the service (create 3 sessions)

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --n-session 3
```

curl Example

```
# Slot 0 conversation
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer no-key" \
    -d '{
        "messages": [{"role": "user", "content": "Hello, my name is Alice"}],
        "id_slot": 0
    }'

# Slot 1 conversation (independent context, unaware of Slot 0's conversation)
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer no-key" \
    -d '{
        "messages": [{"role": "user", "content": "Hello, my name is Bob"}],
        "id_slot": 1
    }'
```

4.1.4 Usage Constraints

Per-Slot Independent Context Size

When using `--n-session N` (multi-session mode), each slot can have a different context size (ctx-size) to meet varying memory and context length requirements. Use the `#` delimiter in CLI or JSON arrays in `params.json`.

CLI example (3 slots, different ctx-size):

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin \
    --host 0.0.0.0 --port 8080 --n-session 3 --n-predict 512 \
    -c 4096#2048#1024
```

params.json example (same configuration, using JSON array):

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "models": [{
        "model": "./qwen2.5-3b.rknn",
        "vocab": "./qwen2.5-3b.tokenizer.gguf",
        "embed": "./qwen2.5-3b.embed.bin",
        "ctx-size": [4096, 2048, 1024],
        "predict": 512,
        "n-session": 3
    }]
}
```

With `"n-session": 3`, slot allocation:

Slot	ctx-size	Use Case
0	4096	Long documents / multi-turn conversations
1	2048	Regular conversations

Slot	ctx-size	Use Case
2	1024	Short Q&A / resource-constrained

If fewer array elements than `n-session`, remaining slots use the scalar default. Scalar values (e.g., `"ctx-size": 4096`) are also supported for backward compatibility, setting the same value for all slots.

> **Note**: Per-slot ctx-size affects NPU memory allocation; larger sizes consume more memory. Configure appropriately based on available memory.

Per-Slot Independent Prediction Length

In multi-session mode, each slot can also have a different prediction token count (n-predict). Use the `#` delimiter in CLI or JSON arrays in `params.json`.

CLI example (3 slots, different n-predict):

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin \
    --host 0.0.0.0 --port 8080 --n-session 3 -c 4096 \
    -n 512#256#-1
```

params.json example (same configuration, using JSON array):

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "models": [{
        "model": "./qwen2.5-3b.rknn",
        "vocab": "./qwen2.5-3b.tokenizer.gguf",
        "embed": "./qwen2.5-3b.embed.bin",
        "ctx-size": 4096,
        "predict": [512, 256, -1],
        "n-session": 3
    }]
}
```

With `"n-session": 3`, slot allocation:

Slot	n-predict	Meaning
0	512	Max 512 tokens generated
1	256	Max 256 tokens generated
2	-1	Set to context length

> **Note**: Each request can further override the current slot's default via the `n_predict` field in `extra_body` for more flexible control.

4.2 Slot Lifecycle

Each slot goes through different lifecycle states during inference. RKLLM3 Server provides pause, resume, and interrupt operations for fine-grained scheduling of inference tasks.

A slot has 5 states:

State	Description
IDLE	Initial state; no task in progress; ready to accept new inference requests
STARTED	Task assigned to slot; initializing
RUNNING	Running; invoking RKNN3 Runtime for inference
PAUSED	Slot paused; KV Cache retained in memory; can resume to pre-pause state
ERROR	Fatal error occurred; slot unusable; requires manual intervention or service restart

4.3 Pause and Resume

4.3.1 Feature Overview

Pause/Resume allows controlling the inference state of a specific slot. Pause suspends inference without destroying the KV Cache; Resume restores from the paused position. Suitable for resource scheduling and dynamic allocation of inference time slices across slots.

4.3.2 API

POST `/slots/{slot_id}/pause`: Pause a slot

Pauses inference on the specified slot; the slot's KV Cache is preserved.

Request

```
POST /slots/{slot_id}/pause
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	No	Model alias; required only when multiple models are loaded

Request body example:

```
{
    "model": "model_alias"
}
```

Response

Success (HTTP 200):

```
{
    "success": true,
    "id_slot": 0,
    "state": "paused"
}
```

Error (HTTP 400):

Error Message	Cause
`Slot is already paused`	The slot is already paused
`Slot is not in a pausable state`	Slot is IDLE or in another non-pausable state
`Invalid slot id: N`	Slot ID out of range

POST `/slots/{slot_id}/resume`: Resume a slot

Resumes inference on a paused slot.

Request

```
POST /slots/{slot_id}/resume
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	No	Model alias; required only when multiple models are loaded

Request body example:

```
{
    "model": "model_alias"
}
```

Response

Success (HTTP 200):

```
{
    "success": true,
    "id_slot": 0,
    "state": "resumed"
}
```

Error (HTTP 400):

Error Message	Cause
`Slot is not paused, cannot resume`	Slot is not currently paused
`Invalid slot id: N`	Slot ID out of range

4.3.3 Example

```
# Pause slot 0
curl -X POST http://localhost:8080/slots/0/pause \
    -H "Content-Type: application/json" \
    -d '{}'

# Resume slot 0
curl -X POST http://localhost:8080/slots/0/resume \
    -H "Content-Type: application/json" \
    -d '{}'
```

4.4 Interrupt

4.4.1 Feature Overview

Interrupt allows directly terminating inference on a specified slot and resetting it to the IDLE state. Unlike Pause, Interrupt immediately terminates inference and releases the slot. The `save_kvcache_path` parameter enables automatic KV Cache saving after interruption (one-step interrupt + save, no separate save call needed). Combined with KV Cache Save/Load, this enables priority-based preemption and other advanced scheduling scenarios.

Typical priority preemption flow:

1. **Start low-priority task**: Launch a long-text inference task on slot 0 in streaming mode
2. **Interrupt and save**: Call `/slots/0/interrupt` with `save_kvcache_path` to interrupt + save KV Cache in one step (state file path auto-derived: `xxx.kv` → `xxx.kv.json`)
3. **Disconnect**: Disconnect the low-priority client; server detects `is_connection_closed()` → `cancel_tasks()` → cleans up residual connections
4. **Execute high-priority task**: Run the high-priority task on slot 0; wait for completion
5. **Restore low-priority task**: Resend the exact same prompt as step 1, with `load_kvcache_path` in `extra_body`; server auto-loads KV cache and state (derived from cache path), resuming from interruption point

4.4.2 API

POST `/slots/{slot_id}/interrupt`: Interrupt a slot

Interrupts inference on the specified slot and resets it to IDLE.

Request

```
POST /slots/{slot_id}/interrupt
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	No	Model alias; required only when multiple models are loaded
`save_kvcache_path`	string	No	If specified, auto-saves KV Cache and inference state after successful interruption (state file path auto-derived: `xxx.kv` → `xxx.kv.json`)

Request body example:

```
{
    "model": "model_alias",
    "save_kvcache_path": "/path/to/kvcache.kv"
}
```

Response

Success (HTTP 200):

```
{
    "success": true,
    "id_slot": 0,
    "state": "idle",
    "kvcache_saved": true,
    "kvcache_path": "/path/to/kvcache.kv"
}
```

If `save_kvcache_path` is not specified, `kvcache_saved` and `kvcache_path` are omitted.

Error (HTTP 400 / 500):

Error Message	Cause
`Invalid slot id: N`	Slot ID out of range
`Failed to interrupt slot`	Interrupt failed (slot not in inference state)

4.4.3 Example

curl Example

```
# Interrupt slot 0
curl -X POST http://localhost:8080/slots/0/interrupt \
    -H "Content-Type: application/json" \
    -d '{}'

# Interrupt slot 0 and auto-save KV Cache
curl -X POST http://localhost:8080/slots/0/interrupt \
    -H "Content-Type: application/json" \
    -d '{"save_kvcache_path": "/data/kvcache/task.kv"}'
```

Python Example (Complete Priority Preemption Flow)

```python
import requests
import json

BASE_URL = "http://127.0.0.1:8080"
SLOT_ID = 0

# Step 1: Start low-priority task (long text, streaming mode)
# Use a subprocess or separate thread to send the request,
# so it can be interrupted after receiving some tokens

# Step 2: Interrupt slot 0 and auto-save KV Cache
# One-step interrupt + save; state file path auto-derived:
#   /data/kvcache/task_low_priority.kv -> /data/kvcache/task_low_priority.kv.json
resp = requests.post(
    f"{BASE_URL}/slots/{SLOT_ID}/interrupt",
    json={
        "model": "",
        "save_kvcache_path": "/data/kvcache/task_low_priority.kv"
    },
    timeout=60
)
print(resp.json())  # {"success": true, "id_slot": 0, "state": "idle",
"kvcache_saved": true, "kvcache_path": "/data/kvcache/task_low_priority.kv"}

# Step 3: Disconnect the low-priority client (kill subprocess / close socket)
# Server detects is_connection_closed() → cancel_tasks() → cleans up residual
connections

# Step 4: Execute high-priority task
resp = requests.post(
    f"{BASE_URL}/v1/chat/completions",
    json={
        "model": "default",
        "messages": [{"role": "user", "content": "Write a short poem about
spring."}],
        "max_tokens": 256,
        "stream": False,
        "id_slot": SLOT_ID
    },
    timeout=120
)
print(resp.json()["choices"][0]["message"]["content"])

# Step 5: Restore low-priority task
# Use the exact same prompt as step 1, with load_kvcache_path in extra_body
# State file auto-derived: /data/kvcache/task_low_priority.kv ->
/data/kvcache/task_low_priority.kv.json
resp = requests.post(
    f"{BASE_URL}/v1/chat/completions",
    json={
        "model": "default",
        "messages": [{"role": "user", "content": "Please write a comprehensive
overview of the history of artificial intelligence..."}],
        "max_tokens": 4096,
        "stream": True,
        "id_slot": SLOT_ID,
        "extra_body": {
            "load_kvcache_path": "/data/kvcache/task_low_priority.kv"
        }
    },
    stream=True,
```

```
    timeout=300
)

# Read streaming output; generation resumes from the interruption point
for line in resp.iter_lines(decode_unicode=True):
    if line and line.startswith("data: "):
        data_str = line[6:]
        if data_str == "[DONE]":
            break
        data = json.loads(data_str)
        content = data["choices"][0]["delta"].get("content", "")
        if content:
            print(content, end="", flush=True)
```

Use Cases

- **Multi-task scheduling**: When a high-priority request arrives, interrupt the current low-priority task, execute the high-priority one, then restore
- **User interruption**: User sends a new question while waiting for a long reply; interrupt old task, handle new question, then restore
- **Resource management**: Dynamically allocate NPU compute, ensuring critical tasks complete first

4.5 Concurrency Scheduling Mechanism

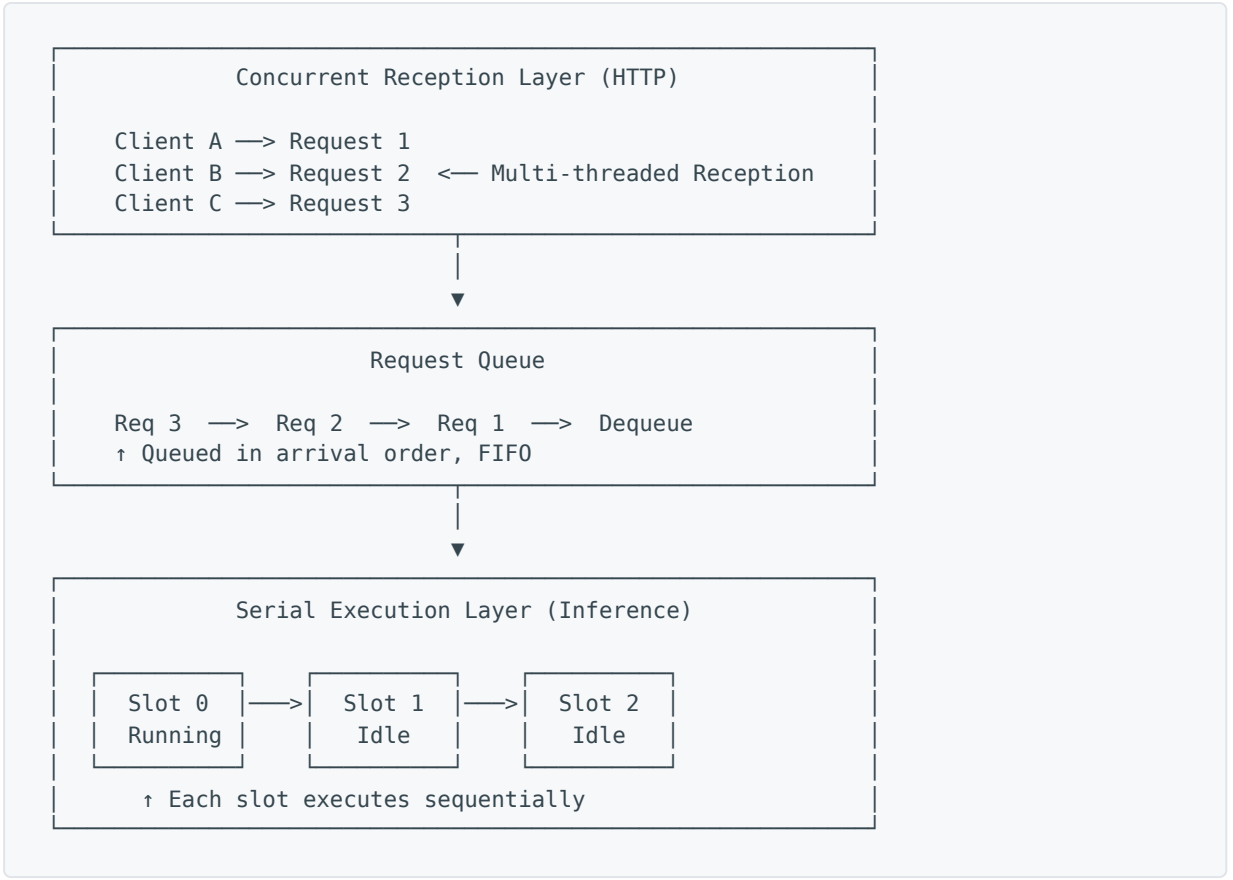
4.5.1 Feature Overview

When `--n-session` is set greater than 1, RKLLM3 Server creates multiple slots, each with independent KV Cache and conversation context. Different slots provide conversation isolation, and the server can accept multiple client requests concurrently.

4.5.2 Concurrency Scheduling Model

RKLLM3 Server's concurrency scheduling model can be summarized as: **HTTP reception is concurrent, but inference execution is serialized via a queue**.

Specifically, the processing flow has two layers:



- **Concurrent Reception Layer**: The HTTP server can simultaneously receive requests from multiple clients; new requests are not blocked by ongoing inference tasks.
- **Serial Execution Layer**: All inference requests are queued in arrival order and processed sequentially. Each request is assigned to an idle slot, executes inference, and releases the slot upon completion.

This design ensures stable and reliable service: even with concurrent client requests, inference will not fail due to resource contention.

4.5.3 Slot Allocation Rules

- **Unspecified slot**: The system auto-selects the least recently used idle slot, ensuring load balancing
- **Specified slot** (via `id_slot`): The specified slot is prioritized; if busy, the request enters the queue
- Slots with errors are skipped and excluded from allocation
- When all slots are busy, new requests enter the queue automatically rather than returning an error immediately. Once a slot completes inference and is released, the next queued request acquires the slot and begins processing.

> **Note**: Queued requests have no timeout mechanism. If a request is not processed for an extended period, clients should set a reasonable timeout.

4.5.4 Response to Pause/Resume/Interrupt

Pause, Resume, and Interrupt operations bypass the request queue and execute directly, ensuring instant response to control commands. See sections 4.3 and 4.4 for details.

---

5. KV Cache

5.1 Save and Load

5.1.1 Feature Overview

KV Cache save/load enables persisting or restoring a slot's KV Cache to/from a file, enabling conversation KV Cache persistence. Suitable for:

- **Conversation continuation**: Save the current conversation's KV Cache, load it after restarting the server, and continue without re-entering history
- **Warm-up acceleration**: Pre-compute prompts to generate KV Cache files, avoiding repeated prefix computation on cold starts

> **Note**: `save_kvcache_path` and `load_kvcache_path` are per-request `extra_body` parameters and cannot be used together in the same request. Save executes after generation completes; load executes before generation starts. After loading KV Cache, the `messages` in the request are still processed (useful for appending new messages).

5.1.2 API

Via extra_body Parameters

Set the following parameters in `extra_body` of `/v1/chat/completions` requests:

Parameter	Type	Description
`save_kvcache_path`	string	Save KV Cache to file (cannot be used with `load_kvcache_path`)
`load_kvcache_path`	string	Load KV Cache from file (cannot be used with `save_kvcache_path`)

POST `/slots/{slot_id}/kvcache/save`: Save KV Cache for a slot

Saves the KV Cache of the specified slot to a file.

> **Note**: This endpoint requires the target slot to be in IDLE state (no ongoing inference). If the slot is busy, the endpoint returns an error.

Request

```
POST /slots/{slot_id}/kvcache/save
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	No	Model alias or path; required only when multiple models are loaded
`path`	string	Yes	File path for saving KV Cache; ensure the parent directory exists

Request body example:

```
{
    "model": "model_alias",
    "path": "/data/kvcache/backup.kv"
}
```

Response

Success (HTTP 200):

```
{
    "success": true,
    "id_slot": 0,
    "path": "/data/kvcache/backup.kv"
}
```

Error (HTTP 400 / 500):

Error Message	Cause
`'path' is required for kvcache save`	`path` field missing from request
`Invalid slot id: N`	Slot ID out of range
`Failed to save kvcache for slot`	Save failed (slot may be busy, or path invalid)

POST `/slots/{slot_id}/kvcache/load`: Load KV Cache into a slot

Loads KV Cache from a file into the specified slot.

> **Note**: Loading overwrites the target slot's current KV Cache. After loading, the slot inherits the KV Cache state from the file, enabling cross-slot migration or restoration of previous conversation context. This endpoint requires the target slot to be in IDLE state.

Request

```
POST /slots/{slot_id}/kvcache/load
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	No	Model alias or path; required only when multiple models are loaded
`path`	string	Yes	File path for loading KV Cache; ensure the file exists

Request body example:

```
{
    "model": "model_alias",
    "path": "/data/kvcache/backup.kv"
}
```

Response

Success (HTTP 200):

```
{
    "success": true,
    "id_slot": 1,
    "path": "/data/kvcache/backup.kv"
}
```

Error (HTTP 400 / 500):

Error Message	Cause
`'path' is required for kvcache load`	`path` field missing from request
`Invalid slot id: N`	Slot ID out of range
`Failed to load kvcache for slot`	Load failed (slot may be busy, or file does not exist)

5.1.3 Usage Constraints

Error Messages

Error Message	Cause
`'save_kvcache_path' and 'load_kvcache_path' cannot both be set in the same request`	Both parameters set in the same request
`directory for 'save_kvcache_path' does not exist: ...`	Parent directory of save path does not exist
`file for 'load_kvcache_path' does not exist: ...`	File to load does not exist

5.1.4 Examples

Save KV Cache (Python)

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

# Chat and save KV Cache
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "user", "content": "Hello, my name is Alice. Please remember my
name."}
    ],
    extra_body={"save_kvcache_path": "/data/kvcache/user_alice.kv"}
)
print(completion.choices[0].message.content)
```

Load KV Cache (Python)

```python
# Load KV Cache and continue conversation
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "user", "content": "What is my name?"}
    ],
    extra_body={"load_kvcache_path": "/data/kvcache/user_alice.kv"}
)
print(completion.choices[0].message.content)
# Output: Your name is Alice
```

Combined with Multi-Session

```python
# Slot 0: save Alice's KV Cache
client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "Hello, my name is Alice"}],
    extra_body={"id_slot": 0, "save_kvcache_path": "/data/kvcache/slot0.kv"}
)

# Slot 1: save Bob's KV Cache
client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "Hello, my name is Bob"}],
    extra_body={"id_slot": 1, "save_kvcache_path": "/data/kvcache/slot1.kv"}
)

# Later, load each independently
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "What is my name?"}],
    extra_body={"id_slot": 0, "load_kvcache_path": "/data/kvcache/slot0.kv"}
)
print(completion.choices[0].message.content)  # Output: Your name is Alice
```

Standalone API Example (curl)

```bash
# First, complete a conversation on slot 0 (via chat/completions)
# Then save slot 0's KV Cache to a file
curl -X POST http://localhost:8080/slots/0/kvcache/save \
    -H "Content-Type: application/json" \
    -d '{"path": "/data/kvcache/slot0_backup.kv"}'

# Load the saved KV Cache into slot 1 (cross-slot migration)
curl -X POST http://localhost:8080/slots/1/kvcache/load \
    -H "Content-Type: application/json" \
    -d '{"path": "/data/kvcache/slot0_backup.kv"}'

# After loading, continue conversation on slot 1
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "messages": [{"role": "user", "content": "What is my name?"}],
        "id_slot": 1
    }'
```

Typical Use Cases

- **Conversation backup and recovery**: Save KV Cache at key conversation points; reload after server restart
- **Cross-slot context migration**: Migrate slot A's conversation context to slot B, freeing slot A for new tasks
- **Precomputation acceleration**: Pre-compute prompts on a slot, save KV Cache, then load on other slots for reuse

5.2 Checkpoint

5.2.1 Feature Overview

The KV Cache checkpoint feature allows saving KV Cache snapshots during inference (applicable to linear attention or sliding window attention models, e.g., Qwen3.5, Gemma4 series), which can be restored later to continue generation. Primary uses:

- **Multi-turn conversations**: Save KV Cache at key conversation nodes to save prefill time
- **Multi-session scenarios**: Each concurrent slot can have independent checkpoint policies
- **Per-request customization**: Checkpoint parameters can be dynamically adjusted per API request

> **Note**: Checkpoint parameters only take effect for linear attention or sliding window attention models (e.g., Qwen3.5, Gemma4 series) and have no effect on other models. `checkpoint_start_pos` and `checkpoint_interval` values are automatically aligned to 128 by the NPU runtime. `max_checkpoint_count` is automatically clamped to `(ctx_size - checkpoint_start_pos) / checkpoint_interval` if exceeded.

5.2.2 API

Checkpoint parameters can be set at three levels, in descending priority:

1. **Per-request** (highest priority): Set in `extra_body` of `/v1/chat/completions` requests. If checkpoint parameters are not specified in the request, the slot's existing checkpoint settings are preserved (not reset to server defaults)
2. **Server CLI**: `--checkpoint-start-pos`, `--checkpoint-interval`, `--max-checkpoint-count`, `--checkpoint-tail-overwrite` (use `#` delimiter for per-slot values)
3. **Server params.json**: `"checkpoint-start-pos"`, `"checkpoint-interval"`, `"max-checkpoint-count"`, `"checkpoint-tail-overwrite"` (use JSON arrays for per-slot values)

Parameter Reference

Parameter	CLI	JSON (`params.json`)	Per-Request (`extra_body`)	Description
`checkpoint_start_pos`	`--checkpoint-start-pos N` or `N#N#...`	`"checkpoint-start-pos": N` or `[N, N, ...]`	`"checkpoint_start_pos": N`	Token position to start saving checkpoints (≥ 0); auto-aligned to 128
`checkpoint_interval`	`--checkpoint-interval N` or `N#N#...`	`"checkpoint-interval": N` or `[N, N, ...]`	`"checkpoint_interval": N`	Token interval for saving checkpoints (0 = disabled); auto-aligned to 128
`max_checkpoint_count`	`--max-checkpoint-count N` or `N#N#...`	`"max-checkpoint-count": N` or `[N, N, ...]`	`"max_checkpoint_count": N`	Maximum number of checkpoints from start_pos; should be ≤ `(ctx_size - start_pos) / interval`; auto-adjusted if exceeded
`checkpoint_tail_overwrite`	`--checkpoint-tail-overwrite 0|1` or `0|1#...`	`"checkpoint-tail-overwrite": 0|1` or `[0, 1, ...]`	`"checkpoint_tail_overwrite": 0|1`	Tail overwrite mode. If `1`, the last checkpoint acts as a circular overwrite slot while others remain fixed (default: `0`)

All four parameters default to `0`, meaning checkpoint saving is disabled by default.

5.2.3 Examples

Single-Session Example

Set checkpoint parameters at startup:

```
./rkllm3-server -m Qwen3.5-4B.rknn --vocab Qwen3.5-4B.tokenizer.gguf --embed
Qwen3.5-4B.embed.bin \
    --host 0.0.0.0 --port 8080 -c -1 --n-predict 2048 \
    --checkpoint-start-pos 128 --checkpoint-interval 128 --max-checkpoint-count 4 --
checkpoint-tail-overwrite 1
```

Or using `params.json`:

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "log-level": 1,
    "models": [{
        "model": "./Qwen3.5-4B.rknn",
        "vocab": "./Qwen3.5-4B.tokenizer.gguf",
        "embed": "./Qwen3.5-4B.embed.bin",
        "ctx-size": -1,
        "predict": 2048,
        "checkpoint-start-pos": 128,
        "checkpoint-interval": 128,
        "max-checkpoint-count": 4,
        "checkpoint-tail-overwrite": 1
    }]
}
```

Override checkpoint parameters per API request:

```
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "messages": [{"role": "user", "content": "Hello!"}],
        "extra_body": {
            "checkpoint_start_pos": 1024,
            "checkpoint_interval": 512,
            "max_checkpoint_count": 4,
            "checkpoint_tail_overwrite": 1
        }
    }'
```

Multi-Session + Per-Slot Checkpoint Configuration

CLI example (3 slots with different checkpoint configs):

```
./rkllm3-server -m Qwen3.5-4B.rknn --vocab Qwen3.5-4B.tokenizer.gguf --embed
Qwen3.5-4B.embed.bin \
    --host 0.0.0.0 --port 8080 -c -1 --n-session 3 --n-predict 2048 \
    --checkpoint-start-pos 128#256#0 --checkpoint-interval 128#256#0 --max-
checkpoint-count 2#3#0 --checkpoint-tail-overwrite 1#0#0
```

params.json example (same configuration, using JSON arrays):

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "models": [{
        "model": "./Qwen3.5-4B.rknn",
        "vocab": "./Qwen3.5-4B.tokenizer.gguf",
        "embed": "./Qwen3.5-4B.embed.bin",
        "ctx-size": -1,
        "predict": 2048,
        "n-session": 3,
        "checkpoint-start-pos": [128, 256, 0],
        "checkpoint-interval": [128, 256, 0],
        "max-checkpoint-count": [8, 4, 0],
        "checkpoint-tail-overwrite": [1, 0, 0]
    }]
}
```

With `"n-session": 3`, slot allocation:

Slot	`start_pos`	`interval`	`max_count`	`tail_overwrite`	Effect
0	128	128	8	1	High-frequency checkpoints, tail circular overwrite
1	256	256	4	0	Medium-frequency checkpoints
2	0	0	0	0	Checkpoints disabled

If fewer array elements than `n-session`, remaining slots use the scalar default (or `0`). Scalar values (e.g., `"checkpoint-interval": 128`) are also supported for backward compatibility.

---

6. Model Extensions

6.1 LoRA

6.1.1 Feature Overview

RKLLM3 Server supports lightweight fine-tuning via LoRA (Low-Rank Adaptation). A single LoRA weight file can contain multiple adapters. You can also pass `--lora-weight` multiple times (or use a `lora-weights` array in the JSON config) to load multiple independent `.lora_weight` files. All adapters loaded from these files are shared across all sessions (slots), but each session can independently configure the scale of each adapter at runtime.

CLI usage (repeat `--lora-weight`):

```
rkllm3-server -m Qwen2.5-3B-Instruct.rknn \
    --lora-weight adapter_a.lora_weight \
    --lora-weight adapter_b.lora_weight
```

JSON config usage (`lora-weights` array):

```
{
    "models": [{
        "model": "Qwen2.5-3B-Instruct.rknn",
        "lora-weights": [
            "adapter_a.lora_weight",
            "adapter_b.lora_weight"
        ]
    }]
}
```

> Note: `lora-weights` is backward-compatible with the legacy single-file `lora-weight` field. If you only load one file, either works; when both are present, `lora-weights` takes precedence.

By default, all adapter scales are set to 0.

6.1.2 API

GET `/lora-adapters`: Get all LoRA adapters

Returns the list of loaded LoRA adapters. Adapters are added via the `--lora-weight` parameter at startup, e.g., `--lora-weight Qwen2.5-3B-Instruct.lora_weight`. Pass `--lora-weight` multiple times to load multiple independent `.lora_weight` files.

Request

```
GET /lora-adapters
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	No	Model alias; required only when multiple models are loaded
`id_slot`	int	No	Slot to query LoRA configuration for. Available when `--n-session` > 1; default: `0`

Request body example:

```
{
    "model": "Qwen2.5-3B-Instruct",
    "id_slot": 0
}
```

Response

```
[
    {"name": "lora_slot0", "scale": 0.0},
    {"name": "lora_slot1", "scale": 0.0}
]
```

POST `/lora-adapters`: Set LoRA adapters

Sets LoRA adapter scales for a specified session (slot). Note that these scales are overridden by the `lora` field in each request.

To disable an adapter, remove it from the list or set its scale to 0.

Request

```
POST /lora-adapters
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	No	Model alias; required only when multiple models are loaded
`id_slot`	int	No	Slot to update LoRA configuration for. Available when `--n-session` > 1; default: `0`
`lora`	array	Yes	LoRA adapter list; each object contains `name` and `scale` fields

Request body example:

```
{
    "model": "Qwen2.5-3B-Instruct",
    "id_slot": 0,
    "lora": [
        {"name": "lora_slot0", "scale": 0.2},
        {"name": "lora_slot1", "scale": 0.8}
    ]
}
```

All sessions share the same set of adapters loaded from `--lora-weight`, but each session can have its own scale values.

Response

Success (HTTP 200):

```
{
    "success": true
}
```

Error (HTTP 400):

Error Message	Cause
`'lora' must be provided`	`lora` field missing from request body
`'lora' must be an array of objects with 'name' and 'scale' fields`	Invalid `lora` field format
`Error: invalid adapter name`	Non-existent adapter name specified
`Error: invalid id_slot`	Invalid slot ID specified

Per-Request Override

In addition to global settings via the `/lora-adapters` endpoint, LoRA scales can be overridden per inference request via the `lora` field in `extra_body`:

```
{
    "messages": [{"role": "user", "content": "Hello!"}],
    "lora": [
        {"name": "lora_slot0", "scale": 0.5},
        {"name": "lora_slot1", "scale": 1.0}
    ]
}
```

> **Note**: The `lora` field in `extra_body` overrides the scale values set via `/lora-adapters` for that slot.

6.2 Tie Word Embedding

6.2.1 Feature Overview

Tie Word Embedding is a model optimization technique that shares input embedding weights with the output layer, eliminating the need for a separate embed file (`--embed`), saving disk space and memory.

> **Note**: This feature requires the RKNN model to have tie-word support enabled during conversion. Standard models still require the `--embed` file.
>
> If the model was not initialized with `--tie-word-embedding`, per-request `tie_word_embedding` overrides are ignored (forced to 0) and the server logs a warning.

6.2.2 API

CLI Parameter

Parameter	Type	Default	Description
`--tie-word-embedding`	`[0|1]`	`0`	Enable tie word embedding mode; the model must support this feature. When set to `1`, no `--embed` file is needed

Per-Request Override

If tie word embedding mode was enabled at initialization and an embed file was also provided, the tie-word setting can be overridden per API request via `extra_body`:

```
{
    "messages": [{"role": "user", "content": "Hello!"}],
    "tie_word_embedding": 0
}
```

If `tie_word_embedding` is not specified in the request, the server default (from CLI or `params.json`) is used.

6.2.3 Examples

Start the Service

```
# Standard model (requires embed file)
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080

# Tie-word model (no embed file needed)
./rkllm3-server -m Qwen3-VL-2B-llm-tie-word.rknn --weight Qwen3-VL-2B-llm-tie-
word.weight \
    --vocab Qwen3-VL-2B-llm_quant.tokenizer.gguf --tie-word-embedding 1 --host
0.0.0.0 --port 8080 -c 4096
```

Python Per-Request Override

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

# Override tie_word_embedding for this request
completion = client.chat.completions.create(
    model="Qwen3-VL-2B",
    messages=[{"role": "user", "content": "Hello!"}],
    extra_body={"tie_word_embedding": 0}
)
print(completion.choices[0].message.content)
```

6.3 Function Calling

6.3.1 Feature Overview

RKLLM3 Server supports OpenAI-compatible Function Calling (tool calling), allowing the model to recognize and invoke external tools during inference. Jinja templates enable this by default; the model parses tool definitions from the request via the chat template and outputs structured `tool_calls` during generation.

> **Note**: The server sets `parse_tool_calls = true` to parse tool call output only when the client request carries `tools` parameters (and `tool_choice` is not `"none"`). If the client does not send `tools`, the model may still output `<tool_call>` tags, but they will not be parsed into OpenAI-compatible JSON `tool_calls` fields.

6.3.2 API

Function Calling is used via the `/v1/chat/completions` endpoint with the following parameters:

Request Parameters

Parameter	Type	Required	Description
`tools`	array	No	Tool definition list, OpenAI-compatible format
`tool_choice`	string/object	No	Controls tool call behavior. Options: `"auto"` (default), `"none"`, `"required"`, or a specific tool
`parallel_tool_calls`	bool	No	Whether to allow parallel tool calls (default: `true`)

Constraints

- When `tools` is used, the server automatically sets `parse_tool_calls = true` to parse tool call output
- When the request contains `tools`, the server auto-regenerates the system prompt (merging tool definitions); `enable_thinking` can be passed in `chat_template_kwargs` to control thinking mode

Request Body Example

```
{
    "model": "Qwen2.5-3B",
    "messages": [
        {"role": "user", "content": "What is the weather like in Beijing today?"}
    ],
    "tools": [
        {
            "type": "function",
            "function": {
                "name": "get_weather",
                "description": "Get the weather information for a specified city",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "city": {
                            "type": "string",
                            "description": "City name"
                        }
                    },
                    "required": ["city"]
                }
            }
        }
    ],
    "tool_choice": "auto"
}
```

Response

When the model decides to call a tool, the response includes a `tool_calls` field in the message:

```
{
    "id": "chatcmpl-xxx",
    "object": "chat.completion",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": null,
                "tool_calls": [
                    {
                        "id": "call_xxx",
                        "type": "function",
                        "function": {
                            "name": "get_weather",
                            "arguments": "{\"city\": \"Beijing\"}"
                        }
                    }
                ]
            },
            "finish_reason": "tool_calls"
        }
    ]
}
```

6.3.3 Examples

Python Invocation

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the weather information for a specified city",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "City name"}
                },
                "required": ["city"]
            }
        }
    }
]

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "What is the weather like in Beijing
today?"}],
    tools=tools,
    tool_choice="auto"
)

msg = completion.choices[0].message
if msg.tool_calls:
    for tc in msg.tool_calls:
        print(f"Tool: {tc.function.name}, Args: {tc.function.arguments}")
else:
    print(msg.content)
```

Multi-Turn Tool Calling

```python
# Round 1: model returns tool_calls
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "What is the weather like in Beijing
today?"}],
    tools=tools
)

# Pass tool_calls back to the model
messages = [
    {"role": "user", "content": "What is the weather like in Beijing today?"},
    completion.choices[0].message.model_dump(),  # assistant message with tool_calls
    {
        "role": "tool",
        "tool_call_id": "call_xxx",
        "content": '{"temperature": "25°C", "weather": "Sunny"}'
    }
]

# Round 2: model generates final response based on tool result
completion2 = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=messages,
    tools=tools
)
print(completion2.choices[0].message.content)
```

6.4 Embedding Model

6.4.1 Feature Overview

RKLLM3 Server supports embedding models via the `--embedding` parameter. When enabled, it provides an OpenAI-compatible `/v1/embeddings` endpoint for converting text to vector representations, suitable for semantic search, text classification, clustering, and more.

> **Note**: When `--embedding` is enabled, the `/v1/completions` endpoint is unavailable. To use both Embedding and Chat functionality, start multiple service instances.

6.4.2 API

POST `/v1/embeddings`: OpenAI-compatible Embeddings API

See OpenAI Embeddings API documentation.

Request

```
POST /v1/embeddings
```

Request Parameters

Parameter	Type	Required	Description
`model`	string	Yes	Model name or alias

Parameter	Type	Required	Description
`input`	string/array	Yes	Text to embed. Can be a single string or an array of strings. Must not be empty
`encoding_format`	string	No	Return format. Options: `"float"` (default) or `"base64"`
`embd_normalize`	int	No	Normalization method. `2` (default, L2 normalization); other values depend on the model's pool type

Request body example:

```
{
    "model": "Qwen3-Embedding-4B",
    "input": ["The food was delicious", "The service was great"],
    "encoding_format": "float"
}
```

Response

Success (HTTP 200):

```
{
    "object": "list",
    "data": [
        {
            "object": "embedding",
            "index": 0,
            "embedding": [0.123, -0.456, 0.789, ...]
        },
        {
            "object": "embedding",
            "index": 1,
            "embedding": [-0.234, 0.567, -0.890, ...]
        }
    ],
    "model": "Qwen3-Embedding-4B",
    "usage": {
        "prompt_tokens": 10,
        "total_tokens": 10
    }
}
```

If `encoding_format` is `"base64"`, the `embedding` field contains base64-encoded float32 byte arrays.

Error Responses

Error Message	Cause
`This server does not support embeddings. Start it with \`--embedding``	Service not started in embedding mode
`"input" or "content" must be provided`	`input` field missing from request
`"content" field is not OAI compatible`	Unsupported `content` field used
`Input content cannot be empty`	Input text is empty

Error Message	Cause
`The format to return the embeddings in. Can be either float or base64`	Invalid `encoding_format` value
`Pooling type 'none' is not OAI compatible`	Model pooling type does not support embedding

6.4.3 Example

Python Invocation

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

# Single text embedding
response = client.embeddings.create(
    model="Qwen3-Embedding-4B",
    input="The food was delicious and the waiter...",
    encoding_format="float"
)
print(response.data[0].embedding)
```

7. Multimodal Input

7.1 Image Input

7.1.1 Feature Overview

RKLLM3 Server supports image input for multimodal models. Pass base64-encoded image data via the `image_url` type in the `content` array.

7.1.2 API

Use `image_url` type content in the `messages` of `/v1/chat/completions` requests:

```
{
    "type": "image_url",
    "image_url": {
        "url": "<base64_image>"
    }
}
```

7.1.3 Example

See 3.2.2 Multimodal Chat Example.

7.2 Audio Input

7.2.1 Feature Overview

RKLLM3 Server supports audio input for multimodal models. Pass base64-encoded audio data via the `input_audio` type in the `content` array.

7.2.2 API

Use `input_audio` type content in the `messages` of `/v1/chat/completions` requests:

```
{
    "type": "input_audio",
    "input_audio": {
        "data": "<base64_audio>",
        "format": "wav"
    }
}
```

7.2.3 Example

See 3.2.2 Multimodal Chat Example.

7.3 Video Input

7.3.1 Feature Overview

For video understanding tasks, RKLLM3 Server supports qwen3_vl deepstack mode video frame input. Use the `video_frames` content type with a `frames` array of base64-encoded video frames. The server processes frames in pairs using the deepstack vision pipeline.

7.3.2 API

Use `video_frames` type content in the `messages` of `/v1/chat/completions` requests:

```
{
    "type": "video_frames",
    "video_frames": {
        "frames": [
            "<base64_frame_1>",
            "<base64_frame_2>"
        ]
    }
}
```

7.3.3 Example

Start the Service

```
./rkllm3-server -m Qwen3-VL-2B-llm.rknn --model2 Qwen3-VL-2B-vision.rknn --vocab
Qwen3-VL-2B-llm.tokenizer.gguf --embed Qwen3-VL-2B-llm.embed.bin --host 0.0.0.0 --
port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --
frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "
<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --img-
width 384 --img-height 384 --video-start "<|vision_start|>" --video-end "
<|vision_end|>" --video-content "<|video_pad|>"
```

Python Example

```python
import base64
from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:8080/v1",
    api_key="sk-no-key-required"
)

def encode_base64(file_path):
    with open(file_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")

# Encode video frames (e.g., extracted from a video at a fixed frame rate)
frame_paths = ["frame_0001.jpg", "frame_0002.jpg", "frame_0003.jpg",
"frame_0004.jpg"]
base64_frames = [encode_base64(fp) for fp in frame_paths]

completion = client.chat.completions.create(
    model="Qwen3-VL-2B",
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "video_frames",
                    "video_frames": {
                        "frames": [
                            f"{b64}"
                            for b64 in base64_frames
                        ]
                    },
                },
                {"type": "text", "text": "Please describe the content of the
video."},
            ],
        }
    ],
    stream=True,
    extra_body={
        "n_predict": 512,
        "id_slot": 0,
    }
)

for chunk in completion:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end='', flush=True)
    if chunk.choices[0].finish_reason == "stop":
        break
print('')
```

7.4 Multimodal Request Format

7.4.1 Request Structure

In multimodal requests, the `content` field of `messages` is an array format supporting mixed content types.

7.4.2 Content Types

Type	Description	Example
`text`	Text content	`{"type": "text", "text": "Please describe the image."}`
`image_url`	Image input (base64)	`{"type": "image_url", "image_url": {"url": "..."}}`
`input_audio`	Audio input (base64)	`{"type": "input_audio", "input_audio": {"data": "...", "format": "wav"}}`
`video_frames`	Video frame input (base64 array)	`{"type": "video_frames", "video_frames": {"frames": ["..."]}}`

7.4.3 Example

See 3.2.2 Multimodal Chat Example.

8. Multi-Card Cascade Inference

8.1 Feature Overview

Multi-Card Cascade Inference runs a model that has already been exported into **K segments (stages)** across **multiple RK182X coprocessors**. The model is pre-split into multiple segment files (`xxx_seg0.rknn` , `xxx_seg1.rknn` , ...), and each segment is loaded onto one RK182X coprocessor. Inference runs in a pipeline order of **stage0 → stage1 → ... → stageK-1**, suitable for scenarios where a single card's memory is insufficient to hold the complete model and multiple cards must be combined.

> **Tip**: For how to export segmented models, refer to the multi-card examples in rknn3-model-zoo.

- **pipeline parallelism**: The output hidden states of the previous segment (stage i) are used as the input of the next segment (stage i+1), passed stage by stage, with the last stage outputting logits. Implementing multi card collaborative inference through pipeline parallelism while improving prefill performance.
- **Bucket processing**: Prefill is split into multiple token buckets and processed bucket by bucket according to `bucket-size` (default 128).
- **KV Cache**: KV Cache save/load is also supported in multi-card mode (automatically split by segment).

8.2 Configuration Parameters

The main multi-card configuration parameters are listed below; the rest (such as `--vocab` , `--embed` , `--ctx-size` , `--predict` , sampling parameters, `--n-session` , etc.) are the same as in single-card mode and are not repeated here.

CLI Parameters

Parameter	Description
`--stage-count N`	Number of segments (>1 enables multi-card cascade inference)
`-m, --model FNAME`	First segment model path, naming convention `xxx_seg0.rknn`
`--weight FNAME`	First segment weight path (optional), naming convention `xxx_seg0.weight`
`--device-id STRING`	Device ID for each segment, `#` -separated (e.g. `0001:11:00.0#0003:31:00.0`)
`--bucket-size N`	Tokens per bucket processed in multi-card inference (default 128)
`--rope-tensor FNAME`	Path to the external rope cache safetensors file (e.g. for Qwen3.5 / Gemma4 series models)

Automatic Segment Naming Derivation

Only the first segment (`_seg0`) model/weight path needs to be specified; the rest of the segment paths are derived automatically using the `_segN` suffix based on the `--stage-count` parameter:

- `model_seg0.rknn` → `model_seg1.rknn`, `model_seg2.rknn`, ...
- If the first segment filename does not contain `_segN`, `_segN` will be inserted before the `.rknn` extension (e.g. `model.rknn` → `model_seg1.rknn`), with a warning.
- If a `dev:` prefix is used for device-side paths (see section 2.3.5), the prefix is preserved during segment derivation (e.g. `dev:/data/model_seg0.rknn` → `dev:/data/model_seg1.rknn`).
- `--device-id` may be omitted, and the internal logic will select devices automatically; if `--device-id` is specified, the number of device IDs must be ≥ `--stage-count`.

JSON Configuration (`params.json`)

Basic format of `params.json` for a multi-card inference model (using two-card cascade as an example):

```
{
  "host": "127.0.0.1",
  "port": 8080,
  "timeout": 30,
  "models": {
    "Qwen3.5-9B": {
      "alias": "Qwen3.5-9B",
      "stage-count": 2,
      "model": "Qwen3.5-9B-llm_seg0.rknn",
      "weight": "Qwen3.5-9B-llm_seg0.weight",
      "vocab": "Qwen3.5-9B-llm.tokenizer.gguf",
      "embed": "Qwen3.5-9B-llm.embed.bin",
      "device-id": ["0001:11:00.0", "0003:31:00.0"],
      "bucket-size": 128,
      "ctx-size": 4096,
      "predict": 512,
      "n-session": 1,
      "checkpoint-start-pos": 128,
      "checkpoint-interval": 128,
      "max-checkpoint-count": 4,
      "checkpoint-tail-overwrite": 1,
      "rope-tensor": "Qwen3.5-9B-llm_seg0.safetensors"
    }
  }
}
```

> **Note**: `rope-tensor` specifies the safetensors file of the external rope cache. It is only required for Qwen3.5 series models when external rope is used, but is mandatory for Gemma-4 series models. Exporting segmented models with rknn3-model-zoo generates multiple such files; any one of them can be used.

An example configuration for Gemma-4-12B-it:

```
{
  "host": "127.0.0.1",
  "port": 8080,
  "timeout": 30,
  "models": {
    "gemma-4-12b": {
      "alias": "gemma-4-12b",
      "stage-count": 2,
      "model": "gemma-4-12b-it_seg0.rknn",
      "weight": "gemma-4-12b-it_seg0.weight",
      "vocab": "gemma-4-12b-it.tokenizer.gguf",
      "embed": "gemma-4-12b-it.embed.bin",
      "device-id": ["0001:11:00.0", "0003:31:00.0"],
      "bucket-size": 128,
      "ctx-size": 4096,
      "predict": 512,
      "n-session": 1,
      "checkpoint-start-pos": 128,
      "checkpoint-interval": 128,
      "max-checkpoint-count": 4,
      "checkpoint-tail-overwrite": 1,
      "rope-tensor": "gemma-4-12b-it_seg0.safetensors"
    }
  }
}
```

8.3 Startup Examples

Command-line startup (using two-card cascade as an example)

Qwen3.5-9B (2 segments):

```
./rkllm3-server --stage-count 2 --model Qwen3.5-9B-llm_seg0.rknn \
    --weight Qwen3.5-9B-llm_seg0.weight \
    --vocab Qwen3.5-9B-llm.tokenizer.gguf --embed Qwen3.5-9B-llm.embed.bin \
    --host 127.0.0.1 --port 8080 -c 4096 --n-predict 512 \
    --checkpoint-start-pos 128 --checkpoint-interval 128 \
    --max-checkpoint-count 4 --checkpoint-tail-overwrite 1 \
    --rope-tensor Qwen3.5-9B-llm_seg0.safetensors
```

Gemma-4-12B-it (2 segments):

```
./rkllm3-server --stage-count 2 --model gemma-4-12b-it_seg0.rknn \
    --weight gemma-4-12b-it_seg0.weight \
    --vocab gemma-4-12b-it.tokenizer.gguf --embed gemma-4-12b-it.embed.bin \
    --host 127.0.0.1 --port 8080 -c 4096 --n-predict 512 \
    --checkpoint-start-pos 128 --checkpoint-interval 128 \
    --max-checkpoint-count 4 --checkpoint-tail-overwrite 1 \
    --rope-tensor gemma-4-12b-it_seg0.safetensors
```

Configuration file startup

Use `--params-file` to specify the `params.json` above:

```
./rkllm3-server --params-file params.json
```

> **Note**: When using `--params-file`, all other command-line parameters are ignored; only the parameters in `params.json` take effect.
>
> **Recommendation**: For Qwen3.5 and Gemma4 series models, it is recommended to enable **Checkpoint** in multi-turn conversations, so that previously saved KV Cache snapshots can be reused in later turns, saving prefill time. In multi-card mode, the Checkpoint policy is applied to all segment sessions. See 5.2 Checkpoint for parameter details.

8.4 KV Cache Save & Load (Multi-Card)

In multi-card mode, KV Cache save/load is automatically split by segment, generating an independent KV Cache file for each segment. Given a user path `path.kv`:

- Save: `path.kv` → `path_seg0.kv`, `path_seg1.kv`, ... (`_segN` inserted before `.kv`)
- Load: likewise loaded from `path_seg0.kv`, `path_seg1.kv`, ... into each segment session

Save/Load via API

- `POST /slots/{slot_id}/kvcache/save`: Save the KV Cache of a slot (automatically split into per-segment files)
- `POST /slots/{slot_id}/kvcache/load`: Load the KV Cache of a slot (automatically loaded from per-segment files)

Example:

```
# Save slot 0's KV Cache (generates test_slot_seg0.kv, test_slot_seg1.kv, ...)
curl -X POST http://localhost:8080/slots/0/kvcache/save \
  -H "Content-Type: application/json" \
  -d '{"path": "/data/test_slot.kv"}'

# Load into slot 0 (restored from test_slot_seg0.kv, test_slot_seg1.kv, ...)
curl -X POST http://localhost:8080/slots/0/kvcache/load \
  -H "Content-Type: application/json" \
  -d '{"path": "/data/test_slot.kv"}'
```

Save/Load via request extra_body

Set `save_kvcache_path` or `load_kvcache_path` in the `extra_body` of a `/v1/chat/completions` request (the two cannot be used together); save/load also splits by segment automatically.

For more KV Cache details, see 5.1 Save and Load.

---

9. Grammar Constraints

Grammar (syntax constraint / structured output) restricts the model's output format, so that the model can only generate content conforming to the specified grammar. It is commonly used in scenarios that require a stable model output format (e.g., enumerated values, JSON objects, fixed tags, etc.).

The grammar feature is disabled by default: enable it via the `--grammar-on` parameter, `"grammar-on": true` in the configuration file, or by configuring a valid global default grammar at startup (`--grammar` / `--grammar-file` / `-j` / `-jf` , which enables the feature automatically so `--grammar-on` is not needed) (see 9.4). When disabled, all grammar constraints described in this chapter take no effect, and the grammar constraint engine is not initialized, saving host memory.

9.1 Request Fields and Priority

Use the following fields in the `POST /v1/chat/completions` request body:

Field	Type	Description
`grammar`	string	User-defined EBNF grammar
`json_schema`	object	JSON Schema object
`response_format`	object	OpenAI-compatible: `{"type":"json_object", ...}` or `{"type":"json_schema","json_schema":{"schema":{...}}}`
`structural_tag`	object	Sequence of structural tags (a combination of const_string / regex / json_schema) for constraining complex outputs with fixed tags (e.g., XML tool-call formats); see 9.5 for details
`thinking_grammar`	string	How the `json_schema` constraint takes effect when thinking is enabled: `auto` (default, the server wraps it automatically) / `off` / `manual` ; see 9.7 for details. Not effective for `structural_tag` : `structural_tag` is always sent as-is, with the thinking prefix provided by the caller
`thinking_end`	string	Optional. Overrides the thinking end marker (auto-detected from the model chat template by default)
`thinking_separator`	string	Optional. Overrides the separator between thinking and the final answer (default `"\n\n"`)
`grammar-on`	bool	Optional. Overrides the grammar feature switch per-request (inherits the startup configuration by default; the switch is enabled automatically when a valid global default grammar is configured at startup, see 9.4). The grammar engine is initialized only at startup: when the server is started without the switch, a request-level `"grammar-on": true` cannot build the engine afterwards and all constraint fields are still ignored; when the server is started with the switch, `"grammar-on": false` disables the grammar constraint for that request only. See 9.4 for details

Mutual exclusion rule: the `json_schema` and `grammar` fields cannot be used at the same time; sending both returns the error `Cannot use both json_schema and grammar` .

> Note: when multiple constraint fields are sent together, the effective priority is `json_schema` / `response_format` (JSON Schema) > `structural_tag` > `grammar` (EBNF), and all of them take precedence over the default grammar set at CLI startup.

Usage notes:

- **Grammar constrains the model output process**: the model can only output content that conforms to the constraint; if the model's inference result conflicts with the constraint, the output is forcibly corrected to a legal sequence.
- **The grammar of each request is independent**: the grammar of a request is cleaned up after it finishes and does not affect the next round.
- **There is a one-time cost the first time it is enabled**: the server completes grammar initialization at model load; each request that sets a grammar triggers a recompilation, and frequently switching grammars incurs extra overhead, so enable it only when needed.

9.2 EBNF Grammar Constraint

9.2.1 Introduction

Constrain the model's output format directly by passing a custom EBNF (Extended Backus-Naur Form) grammar via the `grammar` field. Suitable for enumerated values, fixed-format strings, and similar scenarios.

- **The EBNF default root rule is named `root`**: the grammar string must start with `root ::= ...` to define the entry rule.
- **The regex must match the entire output** (from start to end); for example, `[0-9]{4}-[0-9]{2}-[0-9]{2}` denotes a complete date.

9.2.2 API

Request Parameters

Parameter	Type	Required	Description
`grammar`	string	No	User-defined EBNF grammar string, must start with `root ::= ...`

9.2.3 Examples

Request-Level EBNF Constraint

Ask the model to output only `yes` / `no` / `maybe`:

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "is it working?"}],
    "grammar": "root ::= \"yes\" | \"no\" | \"maybe\""
  }'
```

9.3 JSON Schema Structured Output

9.3.1 Introduction

Describe the output structure with a JSON Schema via the `json_schema` field or the OpenAI-compatible `response_format` field. The server converts it into a grammar constraint to ensure the output is a valid JSON object. Suitable for information extraction, structured Q&A, and similar scenarios.

9.3.2 API

Request Parameters

Parameter	Type	Required	Description
`json_schema`	object	No	JSON Schema object
`response_format`	object	No	OpenAI-compatible form: `{"type":"json_object", ...}` or `{"type":"json_schema","json_schema":{"schema":{...}}}`, ultimately equivalent to `json_schema`

9.3.3 Examples

`json_schema`

Ask for a JSON object containing `name` (string) and `age` (integer 0~150):

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "Introduce yourself"}],
    "json_schema": {
      "type": "object",
      "properties": {
        "name": {"type": "string"},
        "age":  {"type": "integer", "minimum": 0, "maximum": 150}
      },
      "required": ["name", "age"]
    }
  }'
```

OpenAI `response_format`

Option A: `json_object` (any valid JSON object):

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "Extract the information"}],
    "response_format": {"type": "json_object"}
  }'
```

Option B: `json_schema` (with a concrete schema):

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "Extract weather information"}],
    "response_format": {
      "type": "json_schema",
      "json_schema": {
        "schema": {
          "type": "object",
          "properties": {
            "city":        {"type": "string"},
            "temperature": {"type": "number"}
          },
          "required": ["city", "temperature"]
        }
      }
    }
  }'
```

9.4 Enabling the Grammar Feature and Configuring a Global Grammar

9.4.1 Introduction

The grammar feature is **disabled by default**; there are two ways to enable it:

- **When disabled**: the server does not initialize the grammar constraint engine (saving about 10MB of host memory), and all constraint fields described in this chapter — `grammar` / `json_schema` / `response_format` / `structural_tag`, etc. — are ignored; the server logs a warning when a request actually carries constraint fields.
- **Explicit enabling**: via the CLI parameter `--grammar-on` at startup, or via the `"grammar-on": true` field of the model object in the configuration file (`--params-file`).
- **Automatic enabling**: when a **valid global default grammar** is configured at startup (`--grammar` / `--grammar-file` / `-j` / `-jf`, any of them), the grammar feature is enabled automatically and `--grammar-on` is no longer needed (configuring default grammar content is an explicit intent to use the feature; the server logs a message about the automatic enabling).
- **Per-request override**: the request body supports an optional boolean `grammar-on` field (inherits the startup configuration by default); see 9.1. Note that the grammar engine is initialized only at startup: when the server is started without the switch, a request-level `"grammar-on": true` cannot build the engine afterwards; when the server is started with the switch, `"grammar-on": false` disables the grammar constraint for that request only.

Besides the enable switch, rkllm3-server also supports setting a global default grammar via CLI parameters at **startup**. When a request does not carry any of the `grammar` / `json_schema` / `response_format` fields, these defaults are used automatically.

9.4.2 API

Enable switch

Method	Parameter	Description
CLI	`--grammar-on`	Enable the grammar feature (boolean flag, no value required)
CLI	`--grammar` / `--grammar-file` / `-j` / `-jf`	Configuring a global default grammar **automatically enables** the grammar feature; `--grammar-on` is not needed
Configuration file (`--params-file`)	`"grammar-on": true`	Written inside the corresponding model object; a JSON boolean (numeric `0` / `1` is also tolerated; invalid values are ignored with a warning)

Per-request override

Parameter	Type	Description
`grammar-on`	bool	Overrides the switch per-request (inherits the startup configuration by default)

CLI parameters for the global default grammar

CLI Parameter	Value	Description
`--grammar GRAMMAR`	string	Pass an EBNF grammar string directly
`--grammar-file FNAME`	file path	Read the EBNF grammar from a file
`-j, --json-schema SCHEMA`	JSON string	Pass a JSON Schema
`-jf, --json-schema-file FILE`	file path	Read a JSON Schema from a file

Override order: request-level `grammar` / `json_schema` / `response_format` > CLI default grammar > no constraint.

> **Note**: the grammar feature is **enabled automatically** when a global default grammar is configured (`--grammar` / `--grammar-file` / `-j` / `-jf` , any of them); the feature stays disabled only when started without `--grammar-on` / `"grammar-on": true` and without any global default grammar. **Note**: `-j` / `-jf` rely on a server-internal conversion from JSON Schema to grammar format, with limited support for external `$ref` . For complex schemas with external `$ref` , pre-convert them to EBNF with a script and use them via `--grammar-file` .

9.4.3 Examples

Example 1: Enable the grammar feature and fix the output to `yes` / `no` (`--grammar` already sets the default grammar, so `--grammar-on` can be omitted; kept here to be explicit):

```
./rkllm3-server -m model.rknn --grammar-on --grammar 'root ::= "yes" | "no"'
```

Example 2: Load a grammar file (`--grammar-file` already sets the default grammar, so `--grammar-on` can be omitted):

```
./rkllm3-server -m model.rknn --grammar-file ./grammars/json.ebnf
```

Example 3: Specify a JSON Schema at startup (`-j` already sets the default grammar, so `--grammar-on` can be omitted):

```
./rkllm3-server -m model.rknn -j '{"type":"object","properties":{"name":
{"type":"string"}},"required":["name"]}'
```

Example 4: Load a JSON Schema file at startup (`-jf` already sets the default grammar, so `--grammar-on` can be omitted):

```
./rkllm3-server -m model.rknn -jf ./schemas/person.json
```

Example 5: Enable via the configuration file (add the `"grammar-on"` field inside the corresponding model object in `params.json`):

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "models": [
        {
            "model": "./Qwen3.5-4B.rknn",
            "vocab": "./Qwen3.5-4B.tokenizer.gguf",
            "grammar-on": true
        }
    ]
}
```

Startup command: `./rkllm3-server --params-file params.json`. Note that when `--params-file` is used, all other command-line arguments are ignored; do not mix `--grammar-on` with `"grammar-on": true` (the configuration file takes precedence).

Example 6: With the server started with the switch enabled, disable the grammar constraint for a single request:

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "is it working?"}],
    "grammar-on": false,
    "grammar": "root ::= \"yes\" | \"no\" | \"maybe\""
  }'
```

> Note: when the server is started with `--grammar-on` (or with a default grammar configured), the request-level `"grammar-on": false` makes the `grammar` field ignored and the model outputs freely; when the server is started with neither a default grammar nor `--grammar-on`, the `grammar` field is ignored even if `"grammar-on": true` is sent (the engine cannot be built at runtime).

9.5 structural_tag Structured Constraint

9.5.1 Introduction

When the output needs a "fixed tag wrapping + partially free content" structure (for example, forcing the model to output a tool call in a fixed-tag format such as `<tool_call><function=tool_name>arguments JSON</function></tool_call>`), neither a standalone `grammar` (EBNF) nor `json_schema` can express it directly. In this case, use the `structural_tag` field: it describes the output structure as a **sequence**, where each element can be a fixed string, a regex, or a JSON Schema.

9.5.2 Syntax Structure

`structural_tag` is a JSON object with the following structure:

```
{
  "type": "structural_tag",
  "format": {
    "type": "sequence",
    "elements": [
      {"type": "const_string", "value": "<fixed string>"},
      {"type": "regex", "pattern": "<regex>"},
      {"type": "json_schema", "json_schema": { "<JSON Schema>" }, "style":
"qwen_xml"}
    ]
  }
}
```

Sequence element types:

Element Type	Field	Description
`const_string`	`value`	Fixed string, emitted verbatim
`regex`	`pattern`	Regex constraining the character sequence matchable at this position
`json_schema`	`json_schema`	Embedded JSON Schema constraining the valid JSON object at this position; optional `style` (e.g., `qwen_xml`) specifies the argument style for tool calls

9.5.3 Examples

Force the model to output a tool call in Qwen XML format `<tool_call><function=tool_name>arguments JSON</function></tool_call>`:

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "Please call the ping tool."}],
    "tools": [
      {
        "type": "function",
        "function": {
          "name": "ping",
          "description": "Return a simple ping result.",
          "parameters": {"type": "object", "properties": {}}
        }
      }
    ],
    "tool_choice": "auto",
    "structural_tag": {
      "type": "structural_tag",
      "format": {
        "type": "sequence",
        "elements": [
          {"type": "const_string", "value": "\n<tool_call>\n<function="},
          {"type": "regex", "pattern": "[A-Za-z0-9_-]{1,64}"},
          {"type": "const_string", "value": ">\n"},
          {"type": "json_schema", "json_schema": {"type": "object",
"additionalProperties": true}, "style": "qwen_xml"},
          {"type": "const_string", "value": "\n</function>\n</tool_call>"}
        ]
      }
    }
  }'
```

> Notes: - `structural_tag` has higher priority than `grammar` (EBNF) and lower priority than `json_schema` / `response_format`. - The `regex` in the sequence must match the actual content at that position (e.g., the tool name); the other fixed strings are emitted verbatim. - The full structure of the Qwen XML native protocol is `<tool_call>\n<function=tool_name>arguments JSON</function>\n</tool_call>`: the opening tag `<tool_call>` must not be omitted, otherwise the output has a closing tag without an opening one and response parsing fails when `tools` is present. - This feature suits tool calls and structured-tag scenarios; for ordinary JSON structured output, prefer `json_schema` / `response_format`.

9.6 Combining Structured Output with Tool Calling

9.6.1 Introduction

When a request also carries `tools` (and `tool_choice` is not `"none"`), the server decides whether to constrain the tool-call output based on the request content.

9.6.2 Combination Rules

Request Content	Grammar Behavior
Only `tools`, no grammar	No syntax constraint; the model freely outputs the tool call and the server parses it
`tools` + `grammar` (EBNF)	Constrain the tool-call output with the EBNF grammar in the request
`tools` + `json_schema` / `response_format`	Constrain the tool-call output with the JSON Schema in the request
`tools` + `structural_tag`	Constrain the tool-call output with the structural tag sequence in the request (recommended for fixed-tag tool-call formats; see 9.5)
`tools` without any grammar in the request	Fall back to the CLI startup parameter `--grammar` / `--grammar-file` (EBNF); if none is set at startup, no constraint is applied

9.6.3 Examples

Force the model to output a valid JSON object during a tool call (using the EBNF grammar from the request):

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "Check the weather in Beijing"}],
    "tools": [{"type":"function","function":{"name":"get_weather","parameters":
{"type":"object","properties":{"city":{"type":"string"}},"required":["city"]}}}],
    "tool_choice": "auto",
    "grammar": "root ::= object"
  }'
```

> Notes: - The `grammar` in the request must be an **EBNF** grammar string. - In the tool-call scenario, using `json_schema` / `response_format` is recommended; they describe JSON structures more directly and provide clearer constraints. - Without a syntax constraint, the structured output of a tool call depends entirely on the model's capability and the server-side parsing, so parsing failure is possible; to guarantee format stability, use a grammar constraint.

9.7 Combining Structured Output with Thinking Mode

9.7.1 Introduction

When thinking mode is enabled, the model output consists of two parts: the thinking process first, then the final answer. The RKNN3 runtime applies the grammar starting from the first generated token and does not auto-detect thinking, so the grammar used with thinking must cover the complete generation flow: `thinking text -> thinking end marker -> separator -> final answer`.

The server provides automatic handling only for `json_schema` / `response_format`: `thinking_grammar` defaults to `auto`, and the server automatically wraps the schema into a Structural Tag covering the complete flow; thinking is unconstrained and the final answer conforms to the schema (see Example 1 for details).

For `structural_tag`, the server **always sends it as-is**, without any wrapping or trimming: the complete flow is written by the caller, with all constraint logic exposed (see Example 2 for details).

Supported scope (important)

Constraint Field	Combined with Thinking
`json_schema`	Supported: with `thinking_grammar=auto` (default), the server automatically wraps the complete flow; see Example 1
`response_format` (`json_object` / `json_schema` types)	Supported, equivalent to `json_schema`
`structural_tag`	Supported: always sent as-is, the thinking prefix must be self-provided (fully self-written; see Example 2 / Example 3)
`grammar` (EBNF)	Not supported

If `grammar` (EBNF) is sent when thinking is enabled, the server logs a warning and the constraint applies to the model's entire output (including the thinking part), which may cause abnormal thinking content and final answer output. To constrain the final answer in thinking scenarios, use `json_schema` or `structural_tag` instead.

9.7.2 The `thinking_grammar` Field

This field only affects `json_schema` / `response_format` and is not effective for `structural_tag`:

Value	Behavior
`auto` (default)	The server detects whether thinking is actually enabled for this request; if enabled, it automatically wraps `json_schema` into a Structural Tag covering the complete flow, leaving the thinking part unconstrained. **No extra configuration is needed.**
`off`	No adjustment; the schema constraint applies to the model's entire output (including the thinking part). Suitable for scenarios where thinking is disabled, or where constraining the entire output is explicitly desired
`manual`	Same as `off` for `json_schema`; the caller handles the complete flow by itself

In `auto` mode, the two optional override fields of `json_schema` (usually not needed):

Field	Default	Description
`thinking_end`	Auto-detected from the model chat template	Thinking end marker, e.g., `</think>`
`thinking_separator`	`"\n\n"`	Fixed separator between the thinking end marker and the final answer

> Note: these two fields only affect the automatic wrapping of `json_schema`. `structural_tag` is sent as-is; the thinking end marker and separator are written directly in the tag elements (see Example 2).

9.7.3 Examples

Example 1: Enable thinking + `json_schema` (`auto` automatic wrapping, streaming)

The user only writes the familiar `json_schema`; everything else is handled automatically. In thinking scenarios, streaming output is recommended so the thinking process can be observed in real time:

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "What is 2 + 2? Think first, then give
the answer"}],
    "json_schema": {
      "type": "object",
      "properties": {"answer": {"type": "integer"}},
      "required": ["answer"]
    },
    "stream": true
  }'
```

In the SSE response, deltas are pushed in order:

1. Thinking phase: `delta.reasoning_content` pushes the thinking text progressively (free generation, not constrained by the schema);
2. After thinking ends: `delta.content` pushes the final answer, guaranteed to be JSON conforming to the schema when concatenated, e.g., `{"answer": 4}`;
3. Finally, `data: [DONE]` is received.

Non-streaming (`"stream": false` or omitted) is also supported with equivalent behavior:

- `reasoning_content`: the model's thinking process, free text, not constrained by the schema;
- `content`: the final answer after thinking ends, guaranteed to be JSON conforming to the schema.

The complete constraint actually sent by the server to the inference engine:

```
thinking text (any) -> </think> -> "\n\n" -> JSON conforming to the schema
```

Explicitly sending `thinking_grammar` / `thinking_end` / `thinking_separator`

The thinking end marker and separator used by the automatic wrapping are auto-detected from the model chat template by default (e.g., `</think>` and `"\n\n"`). To control them explicitly, send these three fields directly in the request:

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "What is 2 + 2? Think first, then give
the answer"}],
    "json_schema": {
      "type": "object",
      "properties": {"answer": {"type": "integer"}},
      "required": ["answer"]
    },
    "thinking_grammar": "auto",
    "thinking_end": "</think>",
    "thinking_separator": "\n\n",
    "stream": true
  }'
```

Field description:

- `thinking_grammar`: `auto` (default, the server wraps automatically; written explicitly in this example, behavior identical to omitting it) / `off` (no wrapping; the schema constraint applies to the entire output) / `manual` (same as `off`, the complete flow is handled by the caller);
- `thinking_end`: overrides the auto-detected thinking end marker, written explicitly as `</think>` in the example; if template detection is inaccurate, change it to the model's actual thinking end marker;
- `thinking_separator`: overrides the separator between the thinking end marker and the final answer (default `"\n\n"`), written explicitly as `"\n\n"` in the example; to change the connection between thinking and the answer, use another string (e.g., `"\n"`).

> Note: these three fields only affect the automatic wrapping of `json_schema` / `response_format`; `structural_tag` is sent as-is, with the thinking end marker and separator written directly in the tag elements (see Example 2).

Example 2: Enable thinking + fully self-written `structural_tag` (all constraint logic exposed)

The caller has full control over the constraint logic: the structural_tag carries its own thinking prefix (any thinking text + thinking end marker + separator) plus the final answer constraint, and the server sends it as-is without any wrapping. The following example uses the Qwen3.5 Native XML tool-call protocol:

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "Please call the ping tool."}],
    "thinking_grammar": "auto",
    "structural_tag": {
      "type": "structural_tag",
      "format": {
        "type": "sequence",
        "elements": [
          {"type": "tag", "begin": "", "content": {"type": "any_text"}, "end": "
</think>"},
          {"type": "const_string", "value": "\n\n"},
          {"type": "const_string", "value": "\n<tool_call>\n<function="},
          {"type": "regex", "pattern": "ping"},
          {"type": "const_string", "value": ">\n"},
          {"type": "json_schema", "json_schema": {"type": "object",
"additionalProperties": true}, "style": "qwen_xml"},
          {"type": "const_string", "value": "\n</function>\n</tool_call>"}
        ]
      }
    },
    "stream": false,
    "max_tokens": 512
  }'
```

Complete constraint flow:

```
thinking text (any) -> </think> -> "\n\n" ->
<tool_call>\n<function=ping>\n{...}\n</function>\n</tool_call>
```

Key points:

- The first 2 elements form the complete thinking flow: the thinking text is unconstrained (`any_text`), ends with `</think>`, followed by the fixed separator `\n\n`; the remaining elements describe the final answer (Qwen3.5 Native XML protocol).
- The thinking end marker and separator are written directly in the tag elements; when the model template's markers differ (the auto-detected value can be queried via `thinking_end`), update these two elements accordingly.
- The thinking process is still returned in `reasoning_content`; the tool-call format only constrains the output after thinking ends.
- JSON-in-XML / Hermes protocols (Qwen3 style `<tool_call>\n{"name":..., "arguments":...}\n</tool_call>`) are also supported: keep the 2 thinking-prefix elements and replace the final answer part with a `triggered_tags` node; see Example 3 for the complete example.

Example 3: Enable thinking + fully self-written `structural_tag` (JSON-in-XML / Hermes)

The 2 thinking-prefix elements are wrapped in the outer `sequence`, and the final answer part is a `triggered_tags` node, outputting tool calls in JSON-in-XML style:

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "Please call the ping tool."}],
    "thinking_grammar": "auto",
    "structural_tag": {
      "type": "structural_tag",
      "format": {
        "type": "sequence",
        "elements": [
          {"type": "tag", "begin": "", "content": {"type": "any_text"}, "end": "
</think>"},
          {"type": "const_string", "value": "\n\n"},
          {
            "type": "triggered_tags",
            "triggers": ["<tool_call>\n"],
            "tags": [
              {
                "type": "tag",
                "begin": "<tool_call>\n{\"name\": \"ping\", \"arguments\": ",
                "content": {"type": "json_schema", "json_schema": {"type": "object",
"additionalProperties": true}},
                "end": "}\n</tool_call>"
              }
            ],
            "at_least_one": true,
            "stop_after_first": true
          }
        ]
      }
    },
    "stream": false,
    "max_tokens": 512
  }'
```

Complete constraint flow:

```
thinking text (any) -> </think> -> "\n\n" -> <tool_call>\n{"name": "ping",
"arguments": {...}}\n</tool_call>
```

Key points:

- The first 2 elements still form the complete thinking flow; a `triggered_tags` node cannot contain leading free text, so the thinking prefix must be wrapped in the outer `sequence` and cannot be written inside the node.
- The fixed name prefix is written in the tag's `begin`, and `content` only constrains the arguments object; with multiple functions, add one tag per function and ensure `triggers` is an unambiguous prefix of the corresponding tag's `begin`.
- `at_least_one` + `stop_after_first`: after thinking ends, exactly one tool_call must be generated.
- No `tools` in the request: the native protocol of the Qwen3.5 template is Native XML (see Example 2), which is inconsistent with the JSON-in-XML output; with `tools` present, the server's response parsing fails and returns 500 (see Note 2). Without `tools`,

`reasoning_content` contains the thinking text and the raw tool call is returned directly in `content`, to be parsed by the caller:

```
<tool_call>
{"name": "ping", "arguments": {}}
</tool_call>
```

9.7.4 Notes

1. **`max_tokens` must reserve space for thinking**: when thinking is enabled, the thinking text also consumes generated tokens; if `max_tokens` is too small, generation is truncated during the thinking phase, resulting in an incomplete final answer. It is recommended to increase it appropriately beyond the usual value (e.g., 512 or more).
2. **The tool-call protocol must match the model**: the tool-call protocol described by structural_tag determines the model output format, while the server's response parsing follows the native protocol of the model chat template. When the two are inconsistent (e.g., using JSON-in-XML format on Qwen3.5), the final answer fails to parse and the server returns 500. Workarounds: (a) do not send `tools`, so the raw output is returned directly in `content` and parsed by the caller (see Example 3 for the complete example); (b) make structural_tag use the format consistent with the model's native protocol (Qwen3.5 uses Option 1 Native XML, Qwen3 uses Option 2 JSON-in-XML).
3. **Combined with tools**: when thinking + `tools` + `json_schema` are sent together, `json_schema` takes effect per the priority in 9.1 and the tool-call format constraint no longer applies. For tool-call format constraints under thinking, use `structural_tag` instead (see Example 2).
4. **structural_tag is sent as-is**: the server does not detect, wrap, or modify `structural_tag`. If the thinking prefix is omitted, the constraint takes effect from the first generated token; the thinking phase is suppressed by the final answer constraint, causing abnormal output.
5. **When the template provides no thinking end marker**: the `auto` automatic wrapping of `json_schema` does not take effect and the original behavior is preserved (the constraint takes effect from the first token, without breaking existing requests); `structural_tag` is unaffected and sent as-is.

Appendix

A. API Endpoint Index

Endpoint	Method	Function	Section
`/health`	GET	Health check	2.4
`/v1/chat/completions`	POST	Chat completions	3.1, 3.2, 3.3
`/v1/models`	GET	Model list	B.1
`/v1/embeddings`	POST	Embeddings	6.4
`/lora-adapters`	GET	Get LoRA adapter list	6.1
`/lora-adapters`	POST	Set LoRA adapter list	6.1
`/slots/{slot_id}/pause`	POST	Pause slot	4.3
`/slots/{slot_id}/resume`	POST	Resume slot	4.3
`/slots/{slot_id}/interrupt`	POST	Interrupt slot	4.4
`/slots/{slot_id}/kvcache/save`	POST	Save KV Cache	5.1
`/slots/{slot_id}/kvcache/load`	POST	Load KV Cache	5.1

B. OpenAI Compatibility

B1. POST `/v1/chat/completions`: Chat Completions

Given a CHATML-format JSON description in `messages`, returns predicted completions. Supports both synchronous and streaming modes. Only models with a chat template can be used with this endpoint. By default, the CHATML template is used.

See OpenAI Chat Completions API documentation.

`extra_body` supported options (non-OpenAI-compatible):

Parameter	Type	Description
`n_predict`	int	Number of tokens to predict for this request
`chat_template_kwargs`	object	Chat template parameters for this request, e.g., `{"enable_thinking": false}` to disable thinking output
`id_slot`	int	Slot ID to use (available when `--n-session` > 1; default: 0)
`save_kvcache_path`	string	Save KV Cache to file (cannot be used with `load_kvcache_path`)
`load_kvcache_path`	string	Load KV Cache from file (cannot be used with `save_kvcache_path`)
`top_k`	int	Top-k sampling
`top_p`	float	Top-p sampling
`temperature`	float	Temperature sampling
`repeat_penalty`	float	Penalize repeated token sequences
`frequency_penalty`	float	Repeat alpha frequency penalty

Parameter	Type	Description
`presence_penalty`	float	Repeat alpha presence penalty
`lora`	array	LoRA adapter list. Each object must contain `name` and `scale` fields. Adapters not specified default to scale `0.0`
`tie_word_embedding`	int	Enable tie word embedding mode for this request (0 or 1). When 1, no separate embed file is needed
`checkpoint_start_pos`	int	Token position to start saving checkpoints (≥ 0); auto-aligned to 128
`checkpoint_interval`	int	Token interval for saving checkpoints (0 = disabled); auto-aligned to 128
`max_checkpoint_count`	int	Maximum number of checkpoints to save
`checkpoint_tail_overwrite`	int	Tail overwrite mode (`0` / `1`). If `1` , the last checkpoint acts as a circular overwrite slot

C. Error Codes

RKLLM3 Server's error response format is compatible with the OpenAI API specification. All error responses are JSON with `code` , `message` , and `type` fields.

C1. Error Type Quick Reference

Error Type	HTTP Status	`type` Field	Description
`ERROR_TYPE_INVALID_REQUEST`	400	`invalid_request_error`	Invalid or malformed request parameters
`ERROR_TYPE_AUTHENTICATION`	401	`authentication_error`	API Key authentication failed
`ERROR_TYPE_PERMISSION`	403	`permission_error`	Access denied (reserved, currently unused)
`ERROR_TYPE_NOT_FOUND`	404	`not_found_error`	Requested resource not found
`ERROR_TYPE_SERVER`	500	`server_error`	Internal server error
`ERROR_TYPE_NOT_SUPPORTED`	501	`not_supported_error`	Requested feature not supported (custom)
`ERROR_TYPE_UNAVAILABLE`	503	`unavailable_error`	Service temporarily unavailable, e.g., model loading (custom)

C2. Common Error Messages and Troubleshooting

HTTP Status	Error Message	Possible Cause	Solution
400	`"input" or "content" must be provided`	Request body missing `input` or `content` field	Check embedding request format

HTTP Status	Error Message	Possible Cause	Solution
400	`Input content cannot be empty`	Input content is empty	Ensure request contains valid text
400	`Invalid slot id`	Slot ID does not exist	Verify the slot ID is correct
400	`Slot is already paused`	Slot is already paused	Only call pause in non-paused state
400	`Slot is not paused, cannot resume`	Slot is not paused	Only call resume when paused
400	`Slot is not in a pausable state`	Slot is IDLE or in another non-pausable state	Only call pause during active inference
400	`'path' is required for kvcache save`	`path` parameter missing	Provide a valid cache path
400	`'path' is required for kvcache load`	`path` parameter missing	Provide a valid cache path
404	`File Not Found`	Requested static file does not exist	Check the file path
500	`Server context not available`	Server not properly initialized	Restart service and check model loading
500	`Failed to pause/resume/interrupt slot`	Slot operation failed	Check slot state and retry
500	`Failed to save/load kvcache for slot`	KV Cache read/write failed	Check disk space and path permissions
501	`This server does not support completions`	Started in embedding mode	Restart without `--embedding`
501	`This server does not support embeddings`	Embedding mode not enabled	Restart with `--embedding`
503	`Loading model`	Model is still loading	Wait for model loading to complete

D. Configuration Summary

D1. Server Configuration

CLI Parameter	JSON (`params.json`)	Default	Description
`--host`	`"host"`	`127.0.0.1`	Listening IP address
`--port`	`"port"`	`8080`	Listening port
`-to, --timeout`	`"timeout"`	`600`	Read/write timeout (seconds)
`--log-level`	`"log-level"`	`0`	Log verbosity level (see section 2.5.1)
`--device-id`	`"device-id"`	-	Device ID for multi-device setups

D2. Model Configuration

CLI Parameter	JSON (`params.json`)	Default	Description
`-m, --model`	`"model"`	-	LLM model path
`--weight`	`"weight"`	-	LLM weight path
`--model2`	`"model2"`	-	Vision model path
`--weight2`	`"weight2"`	-	Vision weight path
`--model3`	`"model3"`	-	Audio model path
`--weight3`	`"weight3"`	-	Audio weight path
`--vocab`	`"vocab"`	-	Vocabulary file path
`--embed`	`"embed"`	-	Embed file path
`--embed-ple`	`"embed-ple"`	-	Gemma4 per_layer_inputs
`--rope-tensor`	`"rope-tensor"`	-	Gemma4 rope cache
`--mel-filter`	`"mel-filter"`	-	Audio mel filters
`--lora-weight`	`"lora-weight"`	-	LoRA weight path
`--decrypt-key, --key`	`"decrypt-key"`	-	LLM encrypted model key package path
`--decrypt-key2, --key2`	`"decrypt-key2"`	-	Vision encrypted model key package path
`--decrypt-key3, --key3`	`"decrypt-key3"`	-	Audio encrypted model key package path
`-c, --ctx-size`	`"ctx-size"`	`4096`	Context size (supports `#` delimiter)
`-n, --predict`	`"predict"`	`-1`	Prediction token count (supports `#` delimiter)
`-a, --alias`	`"alias"`	-	Model alias
`--core-mask`	`"core-mask"`	Auto	LLM NPU core mask
`--core-mask2`	`"core-mask2"`	Auto	Vision NPU core mask
`--core-mask3`	`"core-mask3"`	Auto	Audio NPU core mask
`--img-width`	`"img-width"`	`0`	Input image width
`--img-height`	`"img-height"`	`0`	Input image height
`--chat-template-file`	`"chat-template-file"`	-	Custom Jinja template file
`--embedding`	`"embedding"`	`false`	Whether this is an embedding model
`--tie-word-embedding`	`"tie-word-embedding"`	`0`	Enable tie word embedding
`--reasoning-format`	`"reasoning-format"`	`none`	Reasoning format (`none` / `deepseek`)
`--reasoning`	`"reasoning"`	`auto`	Reasoning toggle (`on` / `off` / `auto`)
`--record-path`	`"record-path"`	-	User request save directory
`--n-session`	`"n-session"`	`1`	Number of concurrent sessions

CLI Parameter	JSON (`params.json`)	Default	Description
`--stage-count`	`"stage-count"`	`1`	Number of segments for multi-card cascade inference
`--bucket-size`	`"bucket-size"`	`128`	Tokens per prefill bucket for multi-card cascade inference